



Generative AI Academy

Why customize?

- Lacks up to date / real time data
- Need it to be trained on proprietary / IP protected / subscription based datasets / documents
- Reduce hallucinations, improve accuracy and response, change the response

Multiple approaches available

- **Prompt Engineering**
Refines model input to guide its output.
- **Fine-tuning**
Fine tune for domain specific reasoning/style
- **Retrieval Augmented Generation (RAG)**
Richer context with augmented knowledge in database (vector, knowledge graph)
- **Agents**
Richer context with Tools

Indexing



Indexing Pipeline

Data for the knowledge is ingested from the source and indexed. This involves steps like splitting, creation of embeddings and storage of data.



Loading

This step involves extracting information from different knowledge sources and loading them into documents.



Splitting

This step involves splitting documents into smaller manageable chunks. Smaller chunks are easier to search and to use in LLM context windows.



Embedding

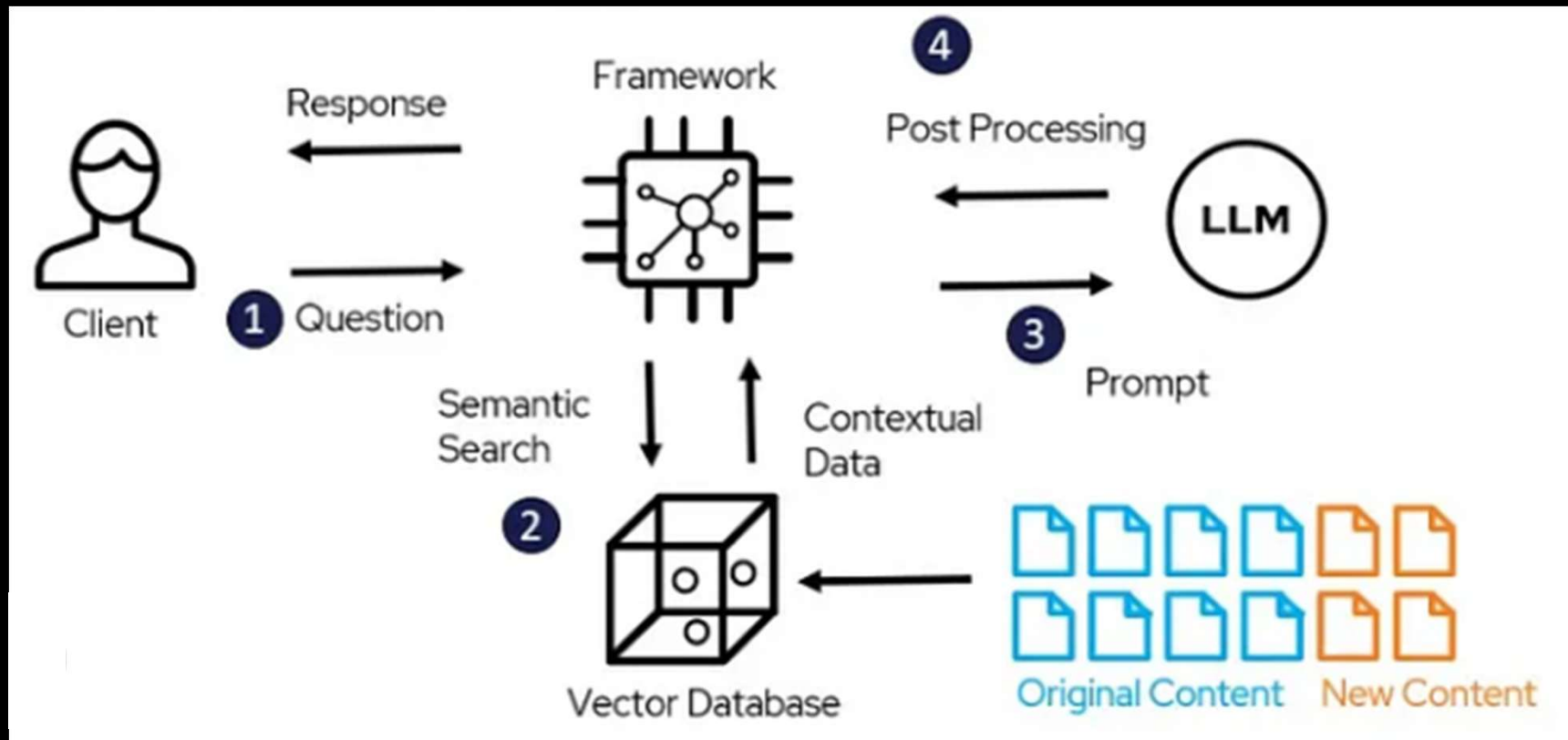
This step involves converting text documents into numerical vectors. ML models are mathematical models and therefore require numerical data.



Storing

This step involves storing the embeddings vectors. Vectors are typically stored in Vector Databases which are best suited for searching.

Retrieval



Popular Embedding Models

word2vec Google's Word2Vec is one of the most popular pre-trained word embeddings. The official paper - <https://arxiv.org/pdf/1301.3781.pdf>

GLOVE The 'Global Vectors' model is so termed because it captures statistics directly at a global level. The official paper - <https://nlp.stanford.edu/pubs/glove.pdf>

fastText Facebook's AI research, fastText builds embeddings composed of characters instead of words. The official paper - <https://arxiv.org/pdf/1607.04606.pdf>

Elmo Embeddings from Language Models, are learnt from the internal state of a bidirectional LSTM. The official paper - <https://arxiv.org/pdf/1802.05365.pdf>

BERT Bidirectional Encoder Representations from Transformers is a transformer bases approach. The official paper - <https://arxiv.org/pdf/1810.04805.pdf>

Criteria to evaluate embedding model?

#weights
#dimensions

Domain?

Popular Vector Databases



Facebook AI Similarity search is a vector index released with a library in 2017



Weaviate is an open source vector database that stores both objects and vectors



Pinecone is one of the most popular managed Vector DB for large scale

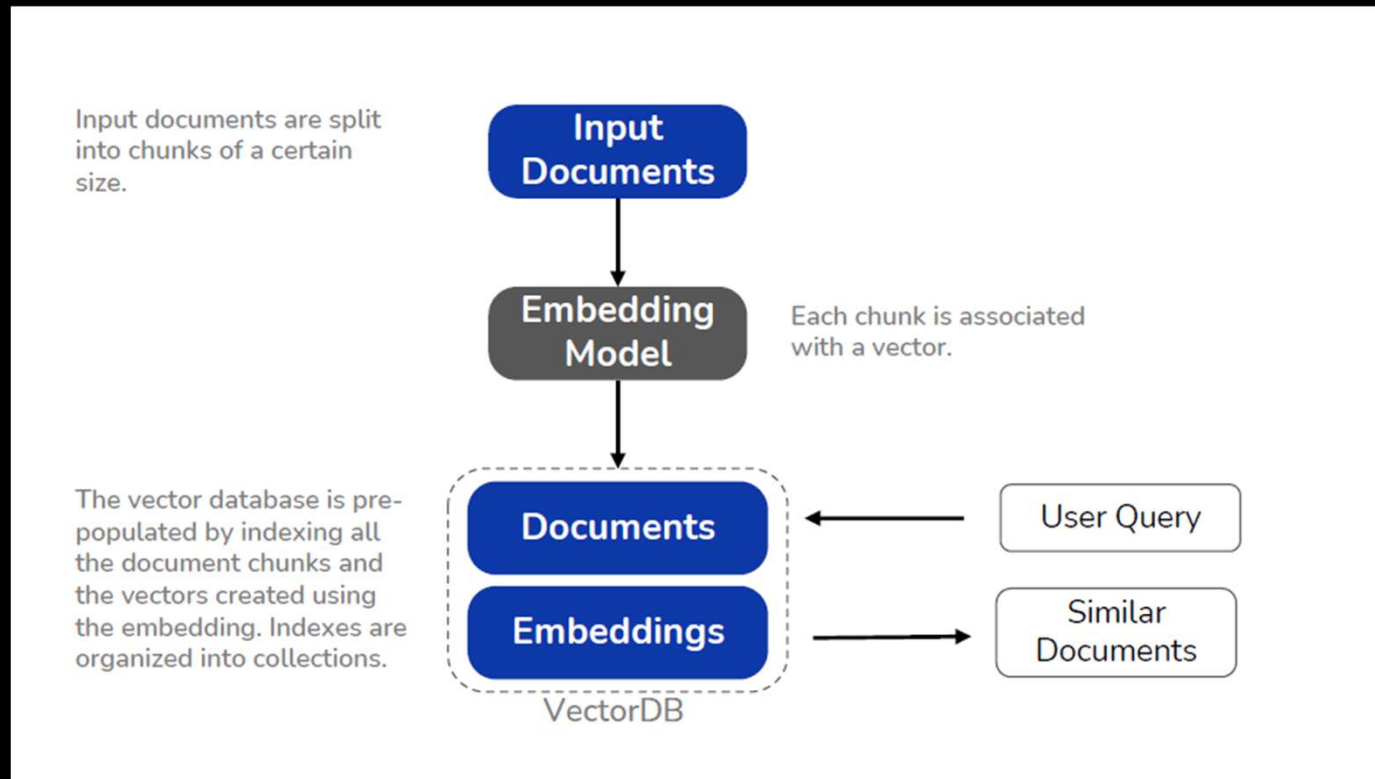


Chromadb is also an open source vector database.

With the growth in demand for vector storage, it can be anticipated that all major database players will add the vector indexing capabilities to their offerings.

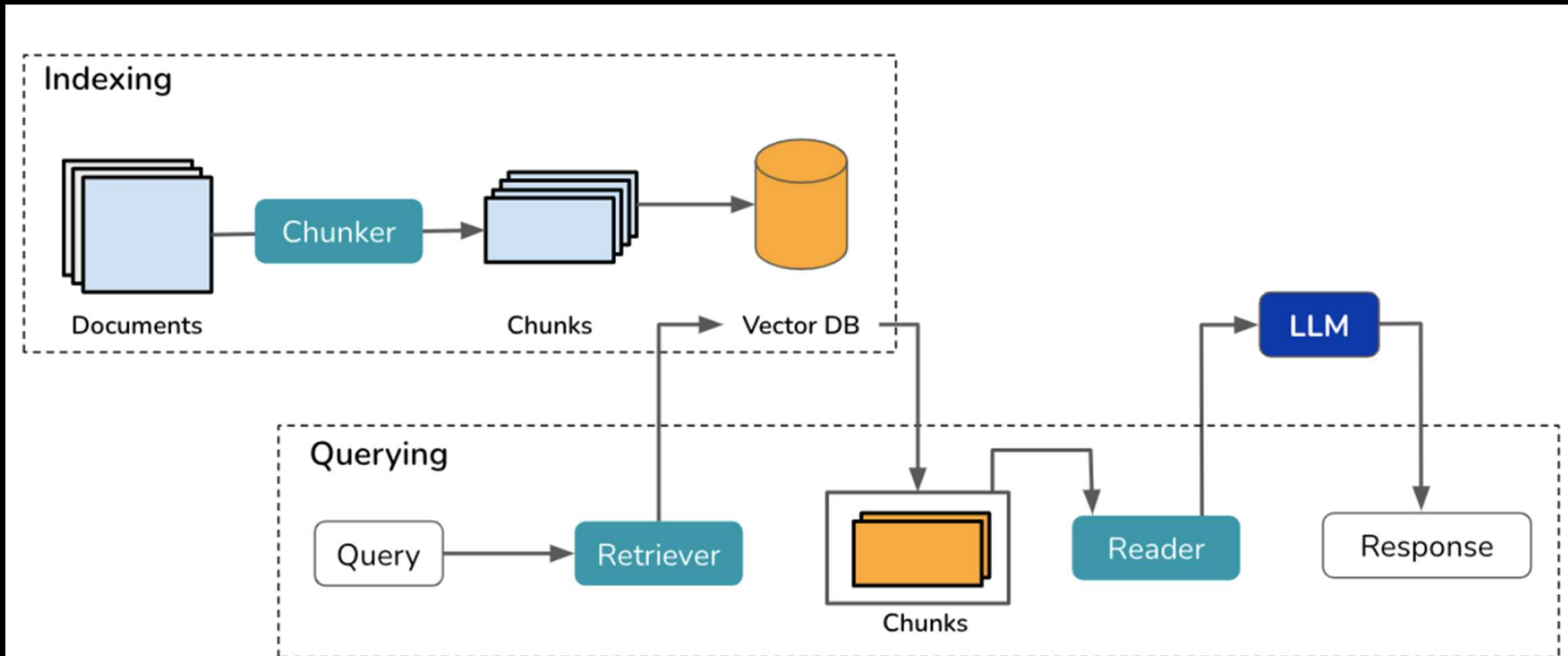
- Qdrant
- Milvus

Vector database

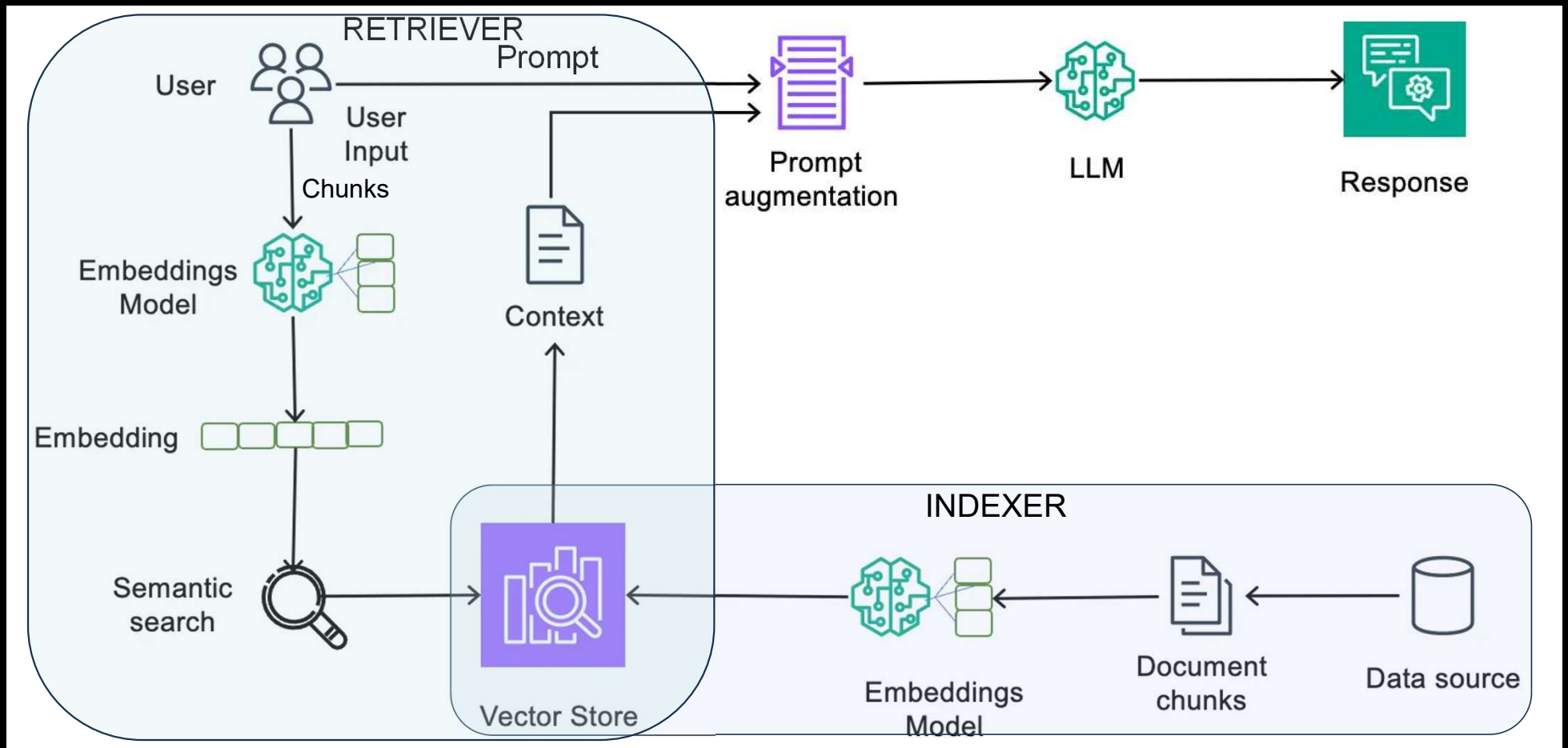


<https://weaviate.io/blog/what-is-a-vector-database>

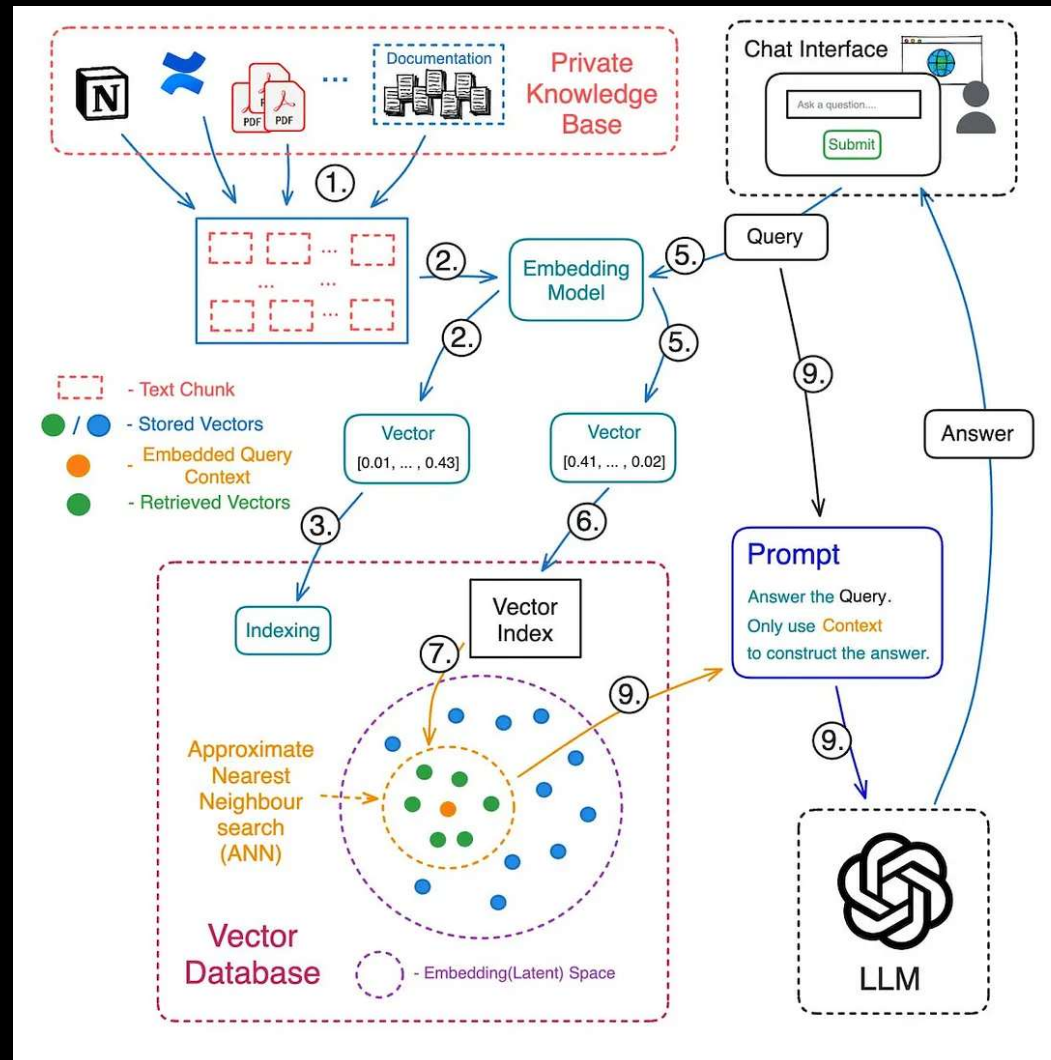
RAG architecture in detail



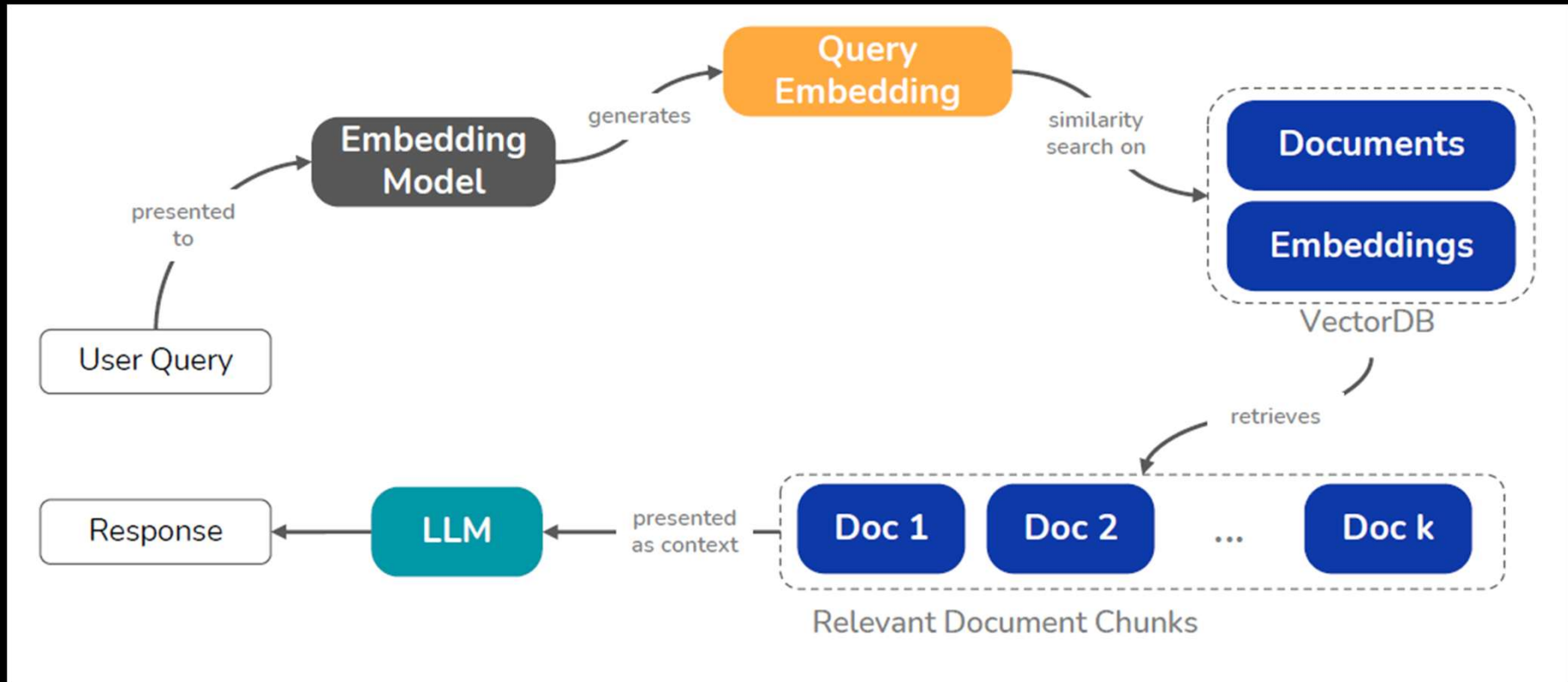
RAG architecture in detail



RAG architecture in detail



Retrieval and generation



Chunking

- What should you be storing in your database?
- Data granularity matters
- Example: Every set of lines Data had, or individual sentences? Or blocks of text of a fixed length? Why just Data's lines?
 - Or maybe summaries of those lines?
 - You can use an LLM for this too...
- The process of splitting up your data prior to storage is referred to as **CHUNKING**.

DATA:
I cannot become nervous. However,
I do sense a certain...
anticipation regarding my role in the wedding.
(beat)
All systems normal, sir.
Sickbay reported that Lieutenant Juarez
went into labor at zero four hundred hours.
We remain on station awaiting the arrival
of the starship Zhukov and guest quarters
have been prepared for Ambassador T'Pel.



I cannot become nervous.

However,
I do sense a certain...
anticipation regarding my role in the wedding.

All systems normal, sir.

Sickbay reported that Lieutenant Juarez
went into labor at zero four hundred hours.

We remain on station awaiting the arrival
of the starship Zhukov and guest quarters
have been prepared for Ambassador T'Pel.

Chunking Strategies

How do we divide text?

Size Based

Divide the text into strings of equal length

- Include overlap on each side of the chunk to include some context
- Easy to implement, but might break up related content into separate chunks

Structure Based

Divide the text based upon the structure (headers, paragraphs, sections)

- Avoids breaking up related content into multiple chunks
- Requires us to understand the structure of the document beforehand

Semantic Based

Divide text into groups of related sentences or sections

- Requires us to understand the meaning of individual sentences
- Computationally expensive, but more relevant chunks

Size based chunking

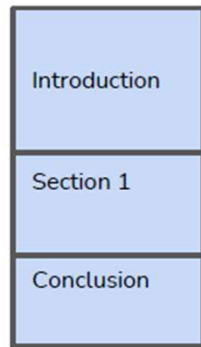
Documents

Could be chunked by

Theme



File

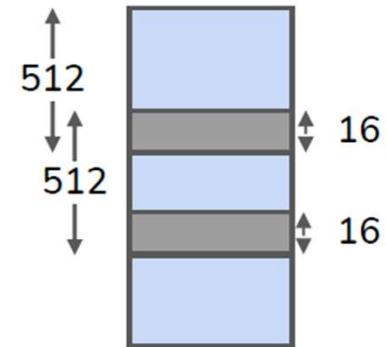


Overlapping
Chunks

Length (Recursive Character Split)



File



Overlapping
Chunks

<https://chunkviz.up.railway.app/>

Structure based chunking

Source Document

Output Chunks

This year our company engaged in many areas of research.

This year our company engaged in many areas of research.

Section 1: Medical Research

Section 1: Medical Research

This year saw significant strides in our understanding of XDR-47, a "bug" we have not seen before.

This year saw significant strides in our understanding of XDR-47, a "bug" we have not seen before.

Section 2: Software Engineering

Section 2: Software Engineering

This division dedicated significant effort to studying various infection vectors in our distributed systems

This division dedicated significant effort to studying various infection vectors in our distributed systems

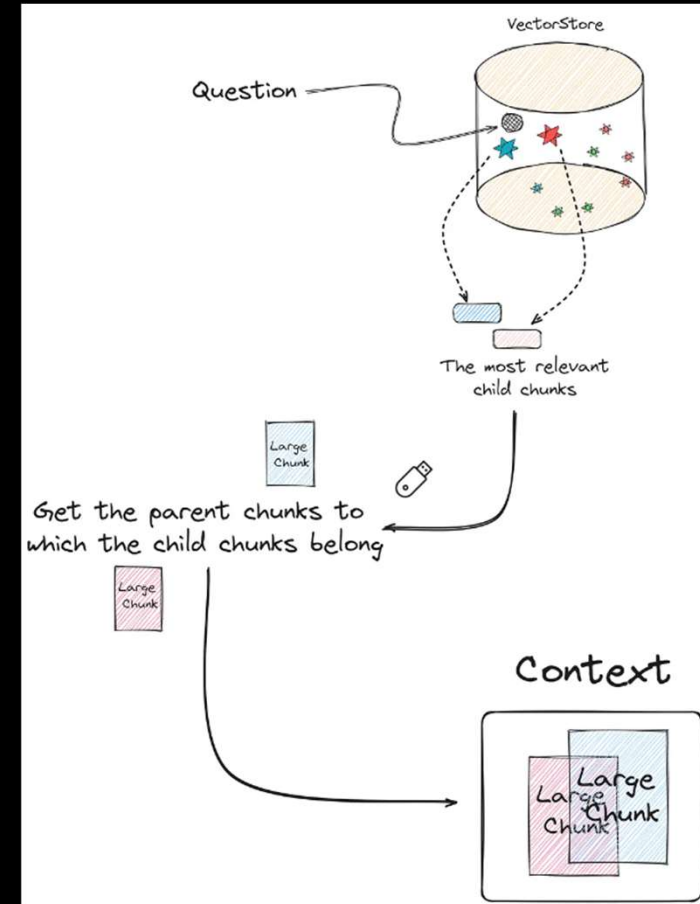
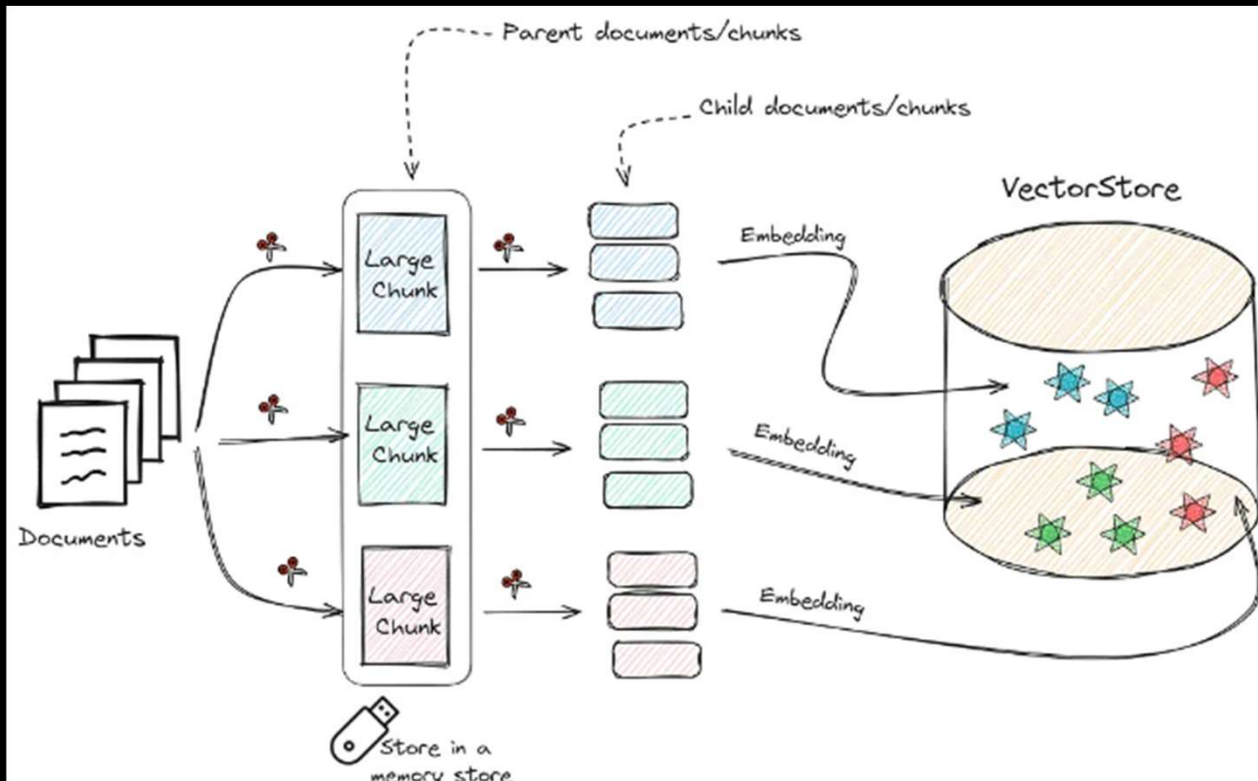
Chunking considerations

Chunk size

If we want to be precise in searching for the most relevant documents, we need to break our documents into **small chunks**.

But it is also very important to provide good context to the LLM, which is achieved by providing **larger chunks**.

Chunking – parent doc retriever



Embedding models

1	Embedding model	Embedding size	Context size	Size (GB)	MTEB Rank (Feb 24)	Release date
2	e5-mistral-7b-instruct	4096	32768	14	4	04/01/2024
3	multilingual-e5-large-instruct	1024	514	1.12	10	08/02/2024
4	BGE-M3	1024	8192	2.27	NA	29/01/2024
5	nomic-embed-text-v1	768	8192	0.55	22	10/02/2024

Indexing

Hierarchical Navigable Small World
(HNSW) algorithm

It is the default Approximate Nearest
Neighbor (ANN) algorithm

Pankaj_cv.pdf

1000 tokens, 100 chunks

chunksize=10tokens

How many embeddings? =1000

1000 tokens, 10 chunks

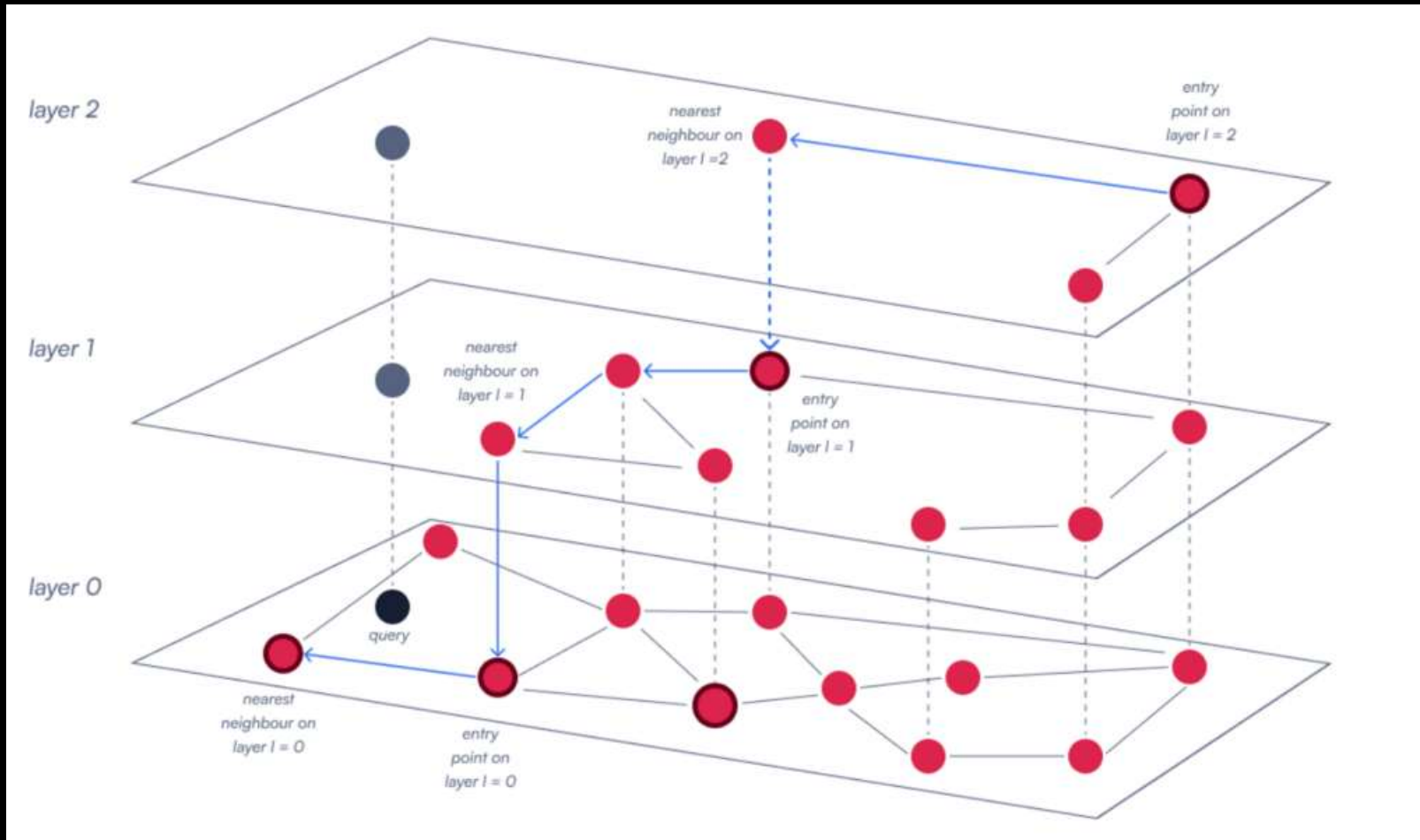
Chunksize=100tokens

How many embeddings? 1000

5 embeddings = 5 chunks



HNSW



<https://qdrant.tech/course/essentials/day-2/what-is-hnsw/>

RAG database

- You could just use whatever database is appropriate for the type of data you are retrieving
 - Graph database (i.e., Neo4j) for retrieving product recommendations or relationships between items
 - Elasticsearch or something for traditional text search (TF/IDF)
 - The functions / tools API can be used to try and get GPT to provide structured queries and extract info from the original query
 - “RAG with a Graph database” in the OpenAI Cookbook is one example
 - https://cookbook.openai.com/examples/rag_with_graph_db
- But for some reason, most examples you find of RAG use a Vector database

Q: 'Which pink items are suitable for children?'

```
{  
  "color": "pink",  
  "age_group": "children"  
}
```

Q: 'Help me find gardening gear that is waterproof'

```
{  
  "category": "gardening gear",  
  "characteristic": "waterproof"  
}
```

Q: 'I'm looking for a bench with dimensions 100x50 for my living room'

```
{  
  "measurement": "100x50",  
  "category": "home decoration"  
}
```

Vector database – how it works?

Let's say you want to search for photos from your vacation,

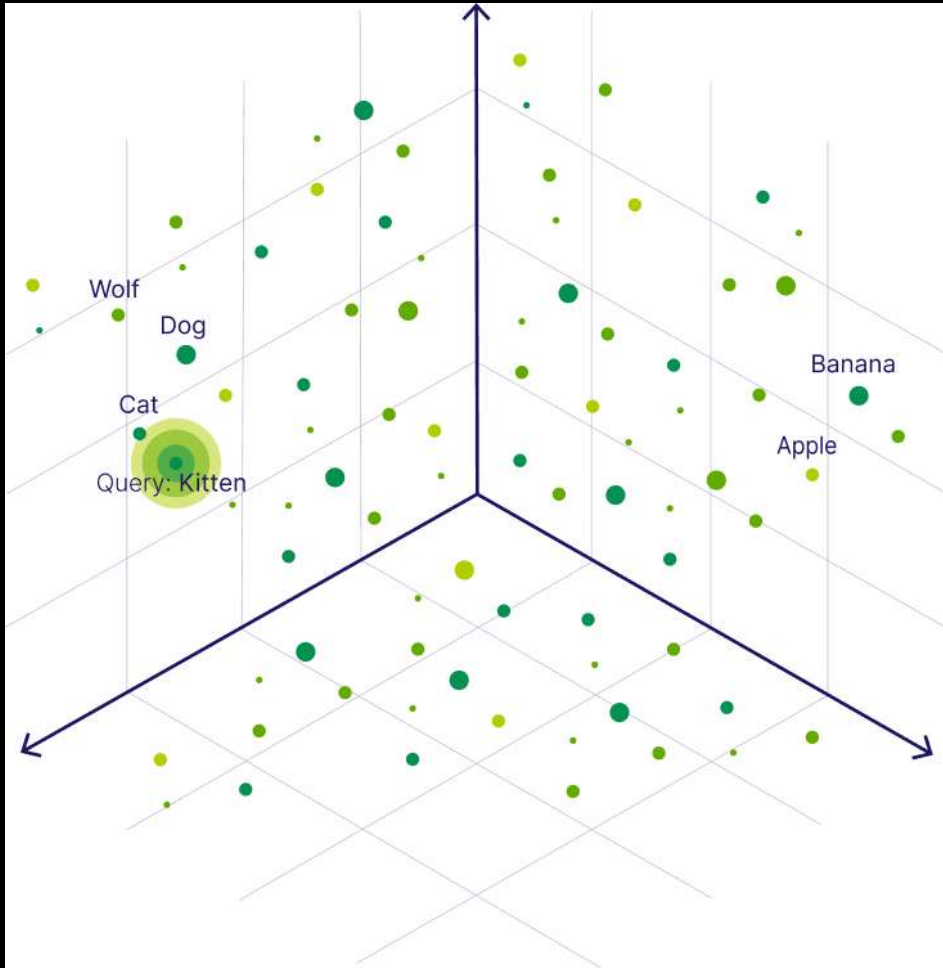
So you type “family vacation in Prague” into the input,

A vector database would turn that text query into a vector and then search for the object vectors **in closest proximity** to the query vector.

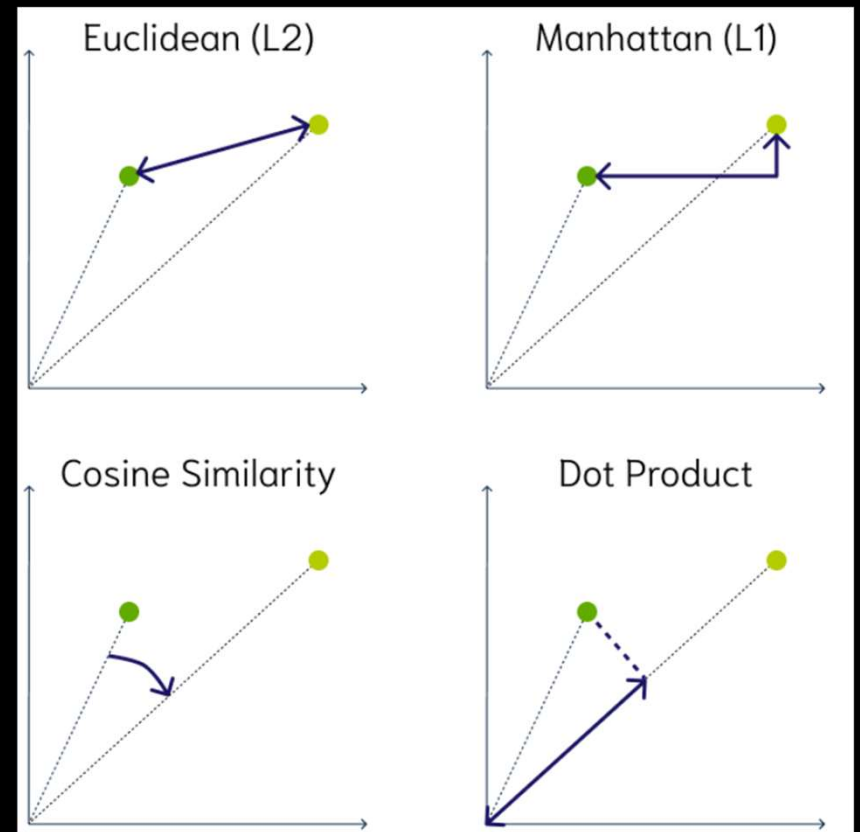
These could be text documents describing the vacation or even pictures you took, and those files are then returned to you.

Search and retrieval

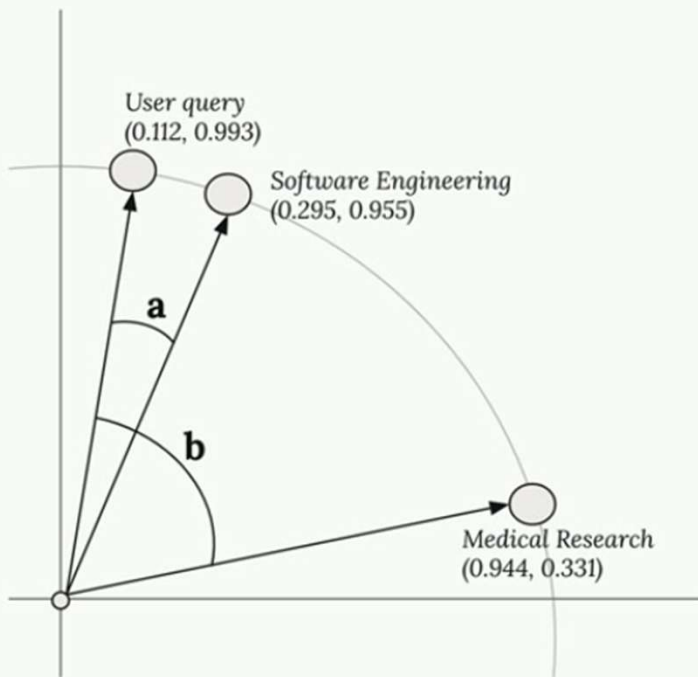
Semantic Search



Distance measurement



Cosine similarity



Cosine Similarity

- Cosine of the angle between the two vectors
- If they're in the exact same direction we'd get 1.0
- If perpendicular, 0.0
- If completely opposite, -1.0

$$\cos(a) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} = \frac{(0.112, 0.993) \cdot (0.295, 0.955)}{\|(0.112, 0.993)\| \cdot \|(0.295, 0.955)\|} = 0.983$$

$$\cos(b) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \cdot \|\mathbf{B}\|} = \frac{(0.112, 0.993) \cdot (0.944, 0.331)}{\|(0.112, 0.993)\| \cdot \|(0.944, 0.331)\|} = 0.398$$

Traditional search Vs semantic search

Traditional Search

```
SELECT question
FROM JeopardyQuestions
WHERE (question LIKE '%dog%' OR
      question LIKE '%cat%' OR
      question LIKE '%wolf%' OR
      (... and so on))
```

VS

Semantic Search

```
{
  Get {
    JeopardyQuestions (
      nearText: { concepts: ["animals"] }
    ) { question }
  }
}
```

BM25 Lexical search

dose-range finding studies. The regulatory pathway, potentially influenced by legal interpretations discussed in Section 5 (Legal Developments), is being carefully mapped. Unlike the broad diagnostic challenges seen in XDR-471 syndrome research (Section 1), CTX-204b benefits from a well-defined target population and biomarker strategy. The team is preparing documentation for regulatory consultation based on these interim results (Report ID: `CTX204b-P2A-INT-01`).

Section 10: Cybersecurity Analysis - Incident Response Report: INC-2023-Q4-011

The Cybersecurity Operations Center successfully contained and remediated a targeted intrusion attempt tracked as `INC-2023-Q4-011`. Threat intelligence indicates the activity aligns with tactics, techniques, and procedures associated with the `ShadowNet Syndicate` threat actor group. Initial access was gained via a spear-phishing email targeting personnel within the finance department, potentially seeking data relevant to Section 3 (Financial Analysis). Endpoint detection and response (EDR) systems flagged anomalous process execution (`PID: 7812`) on workstation `WS-FIN-112`. Subsequent investigation identified malware (`SHA256:e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855`) attempting lateral movement towards server systems and blocking associated command-and-control traffic. Remediation actions included deploying updated endpoint policies (Rule ID: FN7832). This incident highlights the importance of continuous monitoring, proactive vigilance and user training, particularly during high-risk periods.

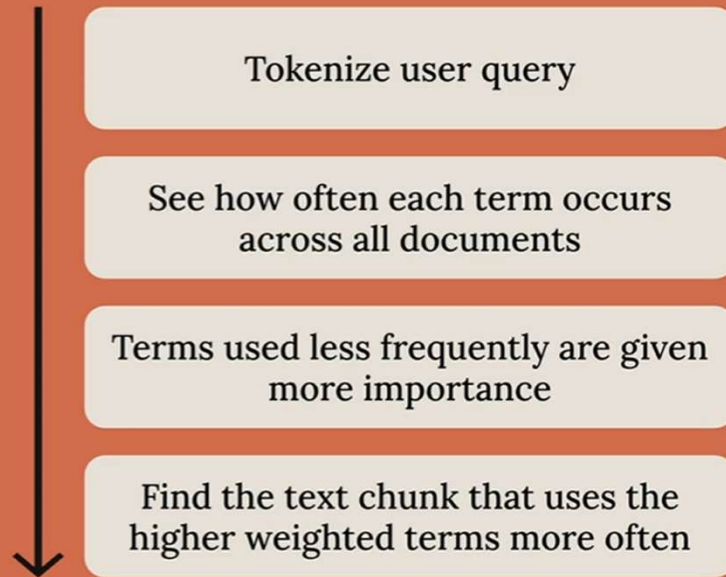
```
# 4. Some time later, a user will ask a question. Generate a user embedding
user_embedding = generate_embedding(
    "what happened with INC-2023-Q4-011?"
)
```

```
# 5. Search the store with the embedding, find the 2 most relevant items
```

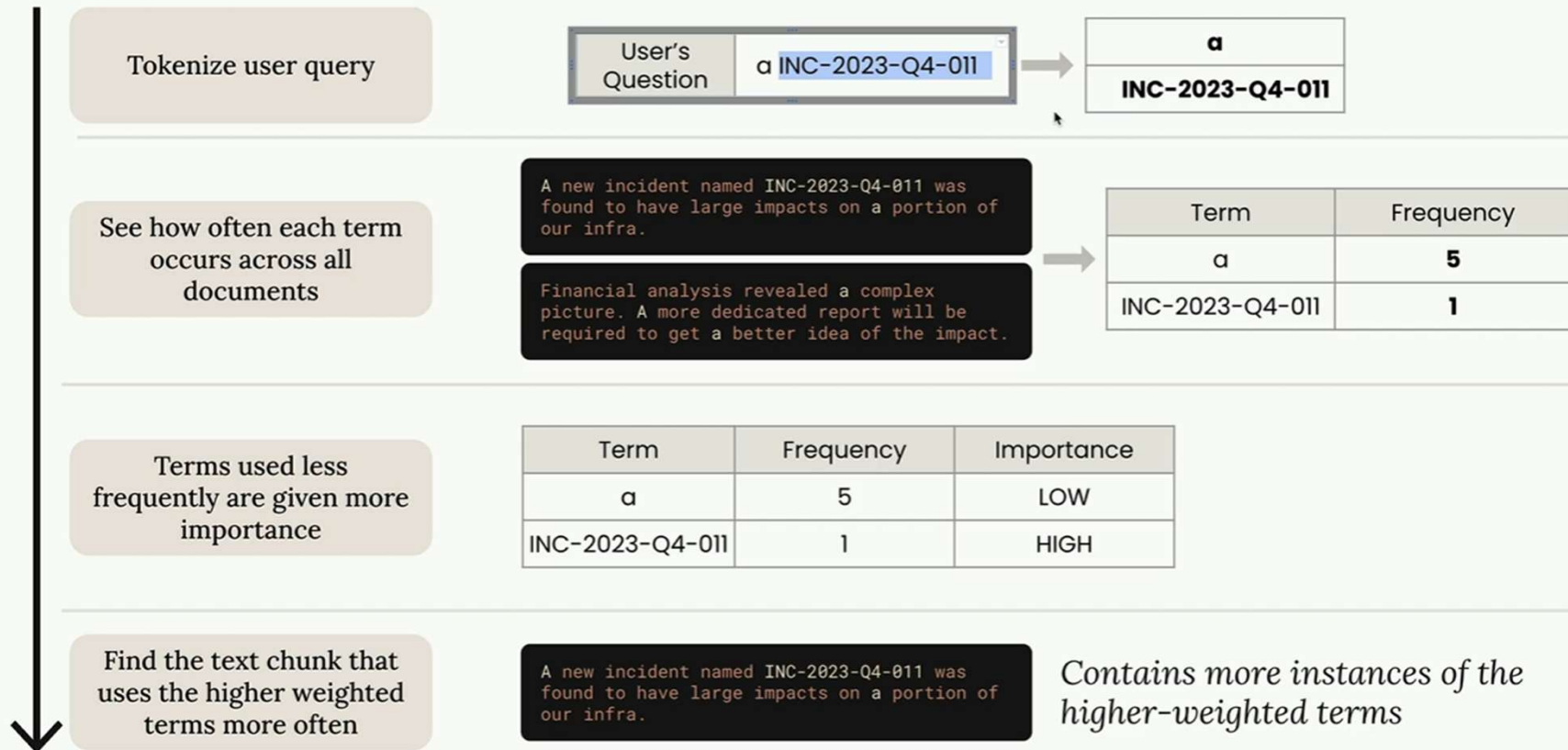
BM25 Lexical search

Lexical Search: BM25

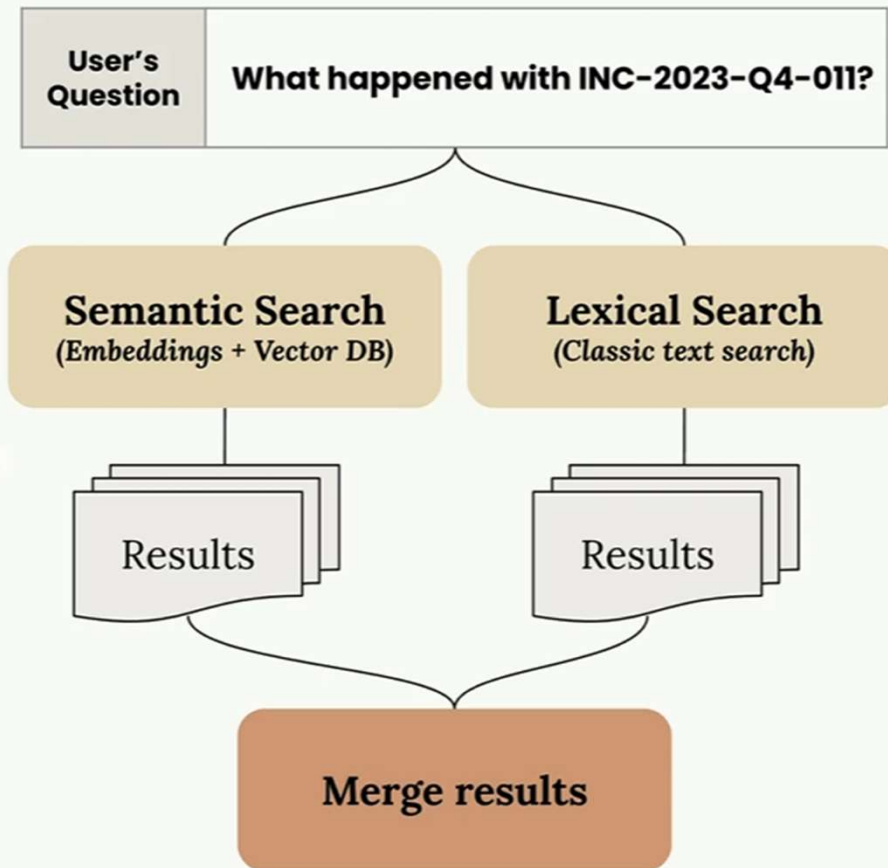
- **Many** methods for implementing text search, but BM25 is commonly used in RAG pipelines.
- **BM25 = Best Match 25**. The '25' means this was the 25th variation of a formula developed by the original creators.



BM25 Lexical search



Combining BM25 with semantic search for better results



Reciprocal Rank Fusion (RRF) to combine the outputs

VectorIndex

Rank	Text Chunk
1	Section 2: Software Engineering...
2	Section 7: Historical Research...
3	Section 6: Product Engineering

BM25Index

Rank	Text Chunk
1	Section 6: Product Engineering
2	Section 2: Software Engineering...
3	Section 7: Historical Research...

Text Chunk	Rank From Vector Index	Rank from BM25 Index
Section 2: Software Engineering...	1	2
Section 7: Historical Research...	2	3
Section 6: Product Engineering...	3	1

Reciprocal Rank Fusion (RRF) to combine the outputs

$$\text{RRF_score}(d) = \sum_{i=1}^n \frac{1}{k + \text{rank}_i(d)}$$

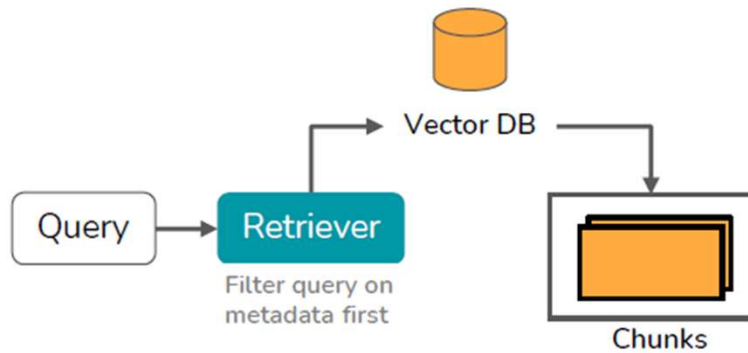
k is a constant, 60 is often used, but going to use 1 here for clearer results

Text Chunk	Rank From Vector	Rank from BM25	Score	Final Rank
Section 2: Software Engineering...	1	2	$1.0 / (1 + 1) + 1.0 / (1 + 2) = \mathbf{0.833}$	1
Section 6: Product Engineering...	3	1	$1.0 / (1 + 3) + 1.0 / (1 + 1) = \mathbf{0.75}$	2
Section 7: Historical Research...	2	3	$1.0 / (1 + 2) + 1.0 / (1 + 3) = \mathbf{0.583}$	3

Higher score = more relevant text chunk

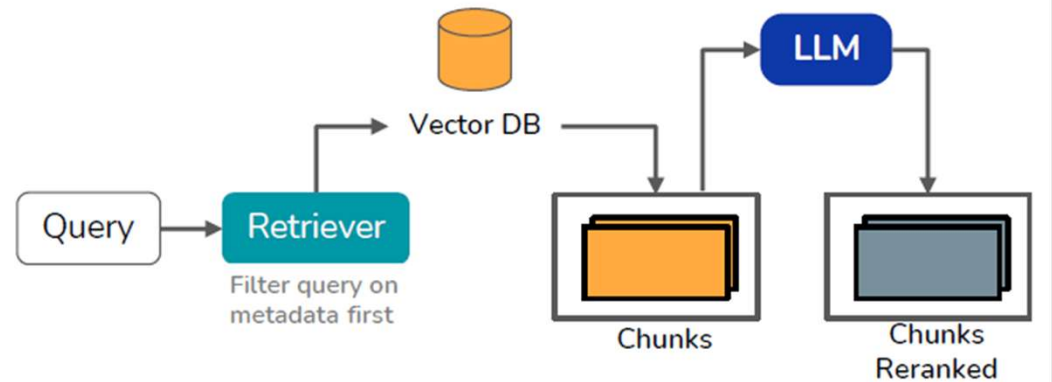
Retrieval techniques

Structured Retrieval



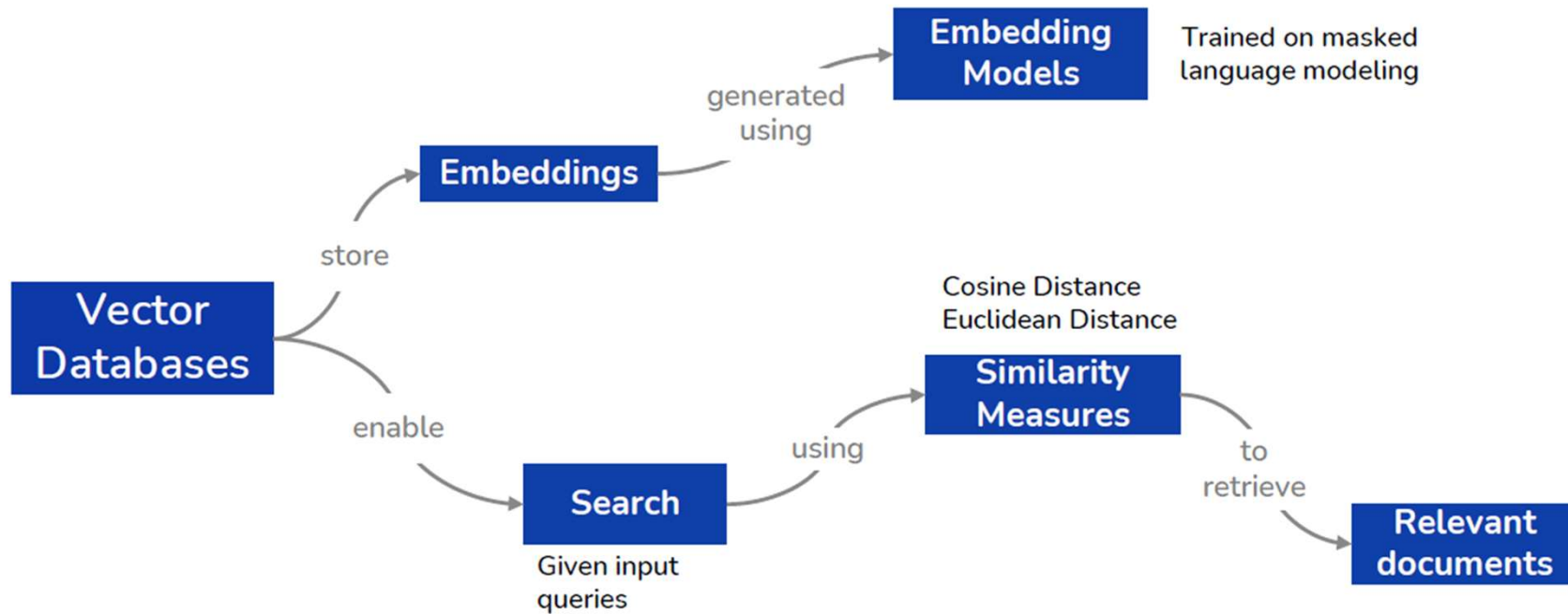
Used when there are many similar documents

Reranking



Used when evaluation reveals poor relevance scores

Vector database – summary of actions



Adding metadata – support citations

- **Step A:** You chunk `doc1.pdf` and `doc2.pdf`, store chunks with `metadata={"source": "doc1.pdf", "page": 2}`, etc.
- **Step B:** When the user asks a question, you embed it, get top-k chunks from the vector store.
- **Step C:** The store returns chunk texts + metadata.

User query:

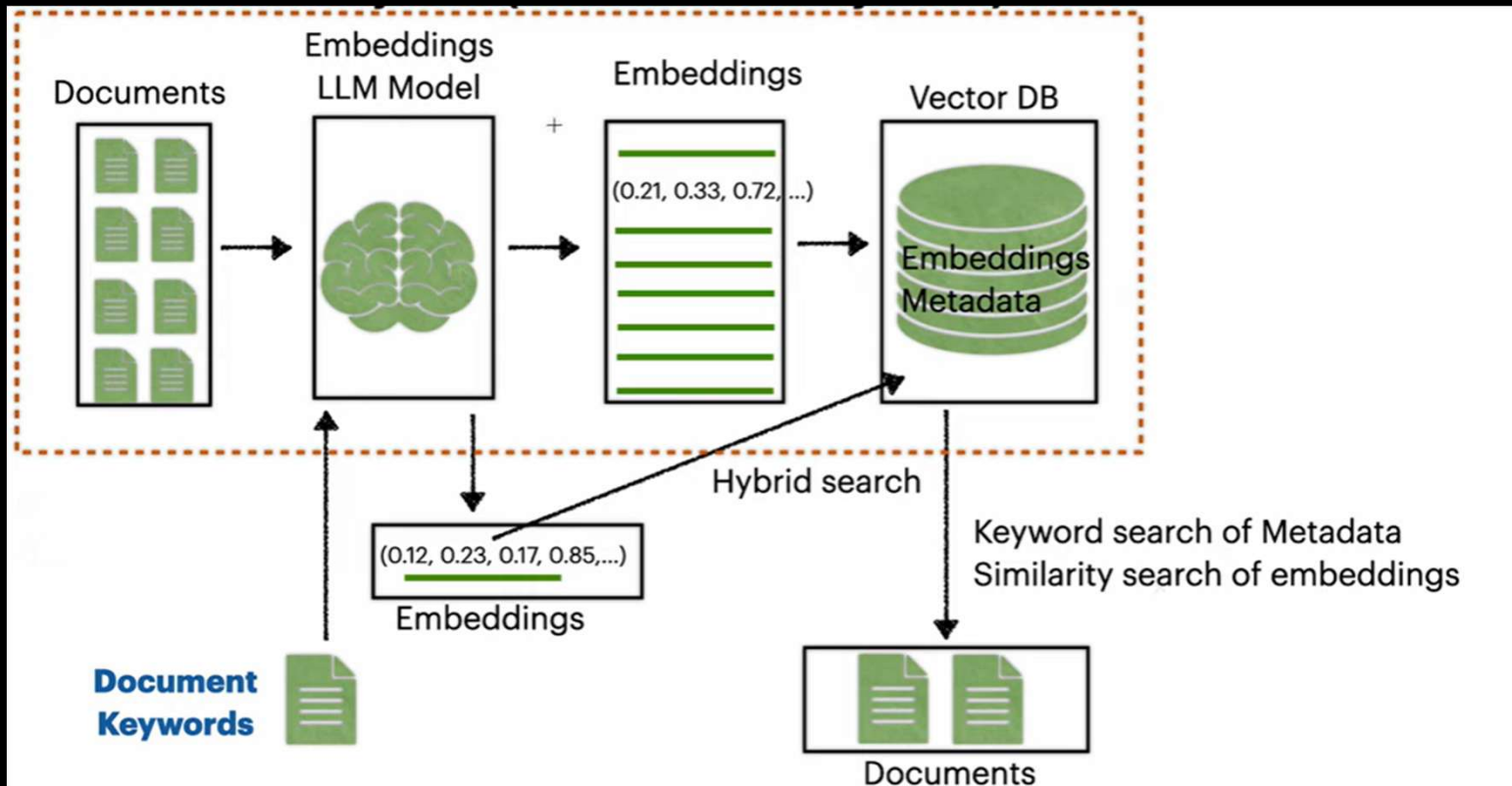
“give me the pricing details for solar panels”

Retrieved Context:

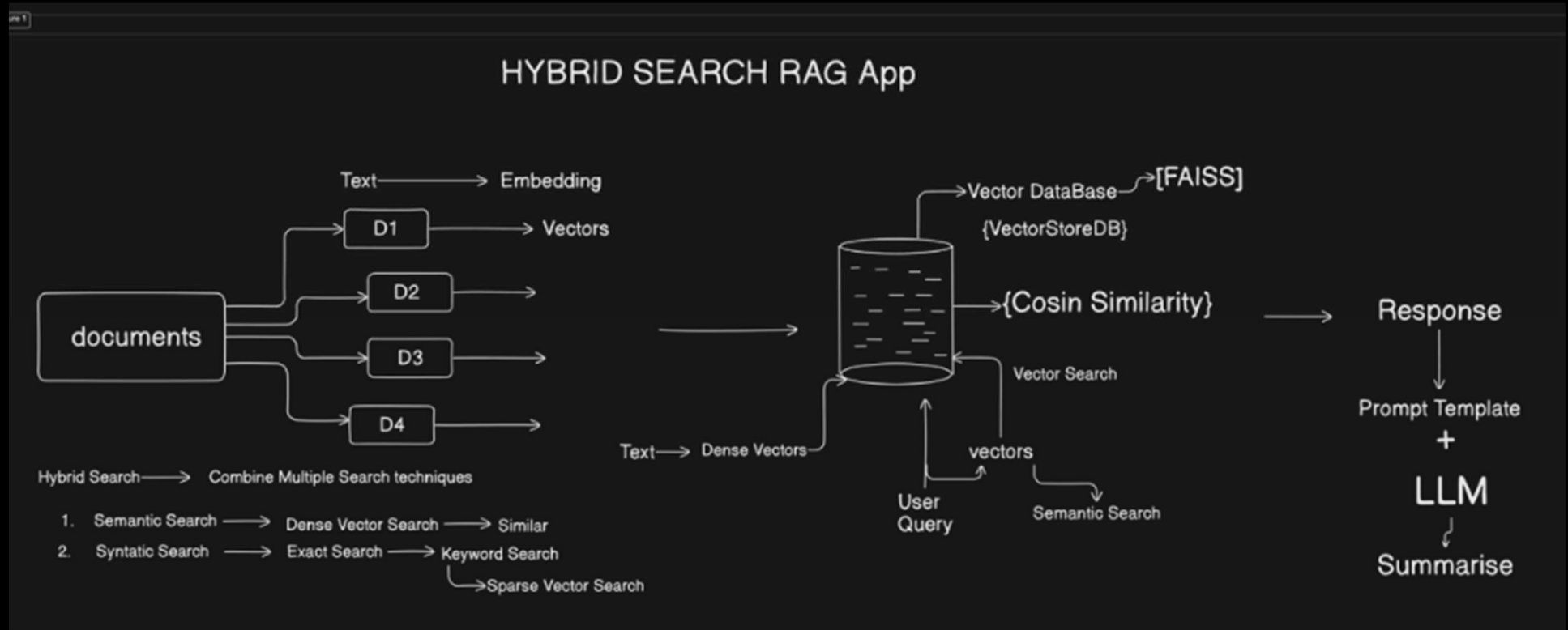
- 1) Source: `doc1.pdf`, Page: 5 "Solar panels cost around \$X per watt..."
- 2) Source: `doc2.pdf`, Page: 3 "Rebates are available in certain states..."

Hybrid Search

Combine Syntactic/keyword search and semantic search

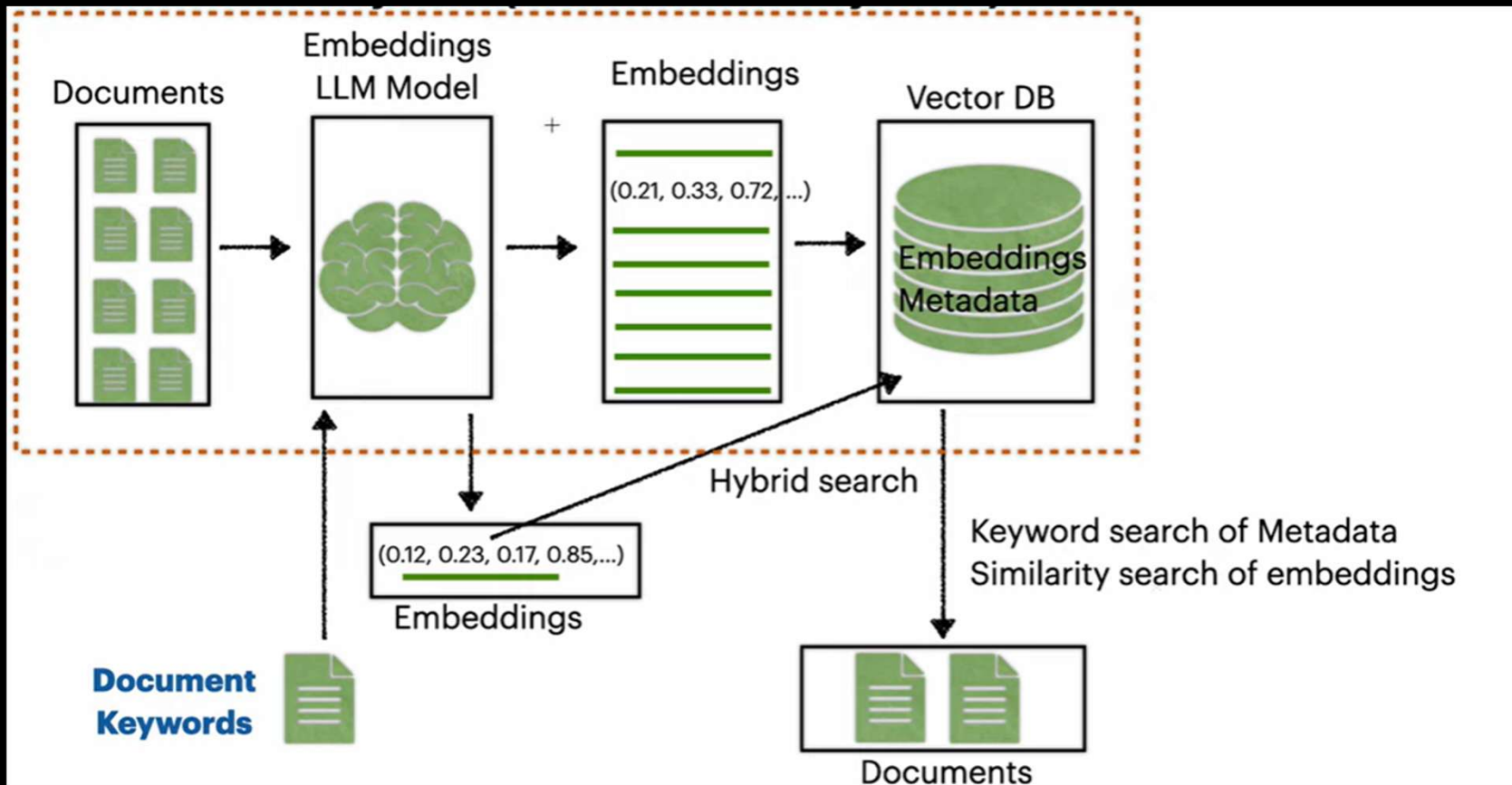


Hybrid Search

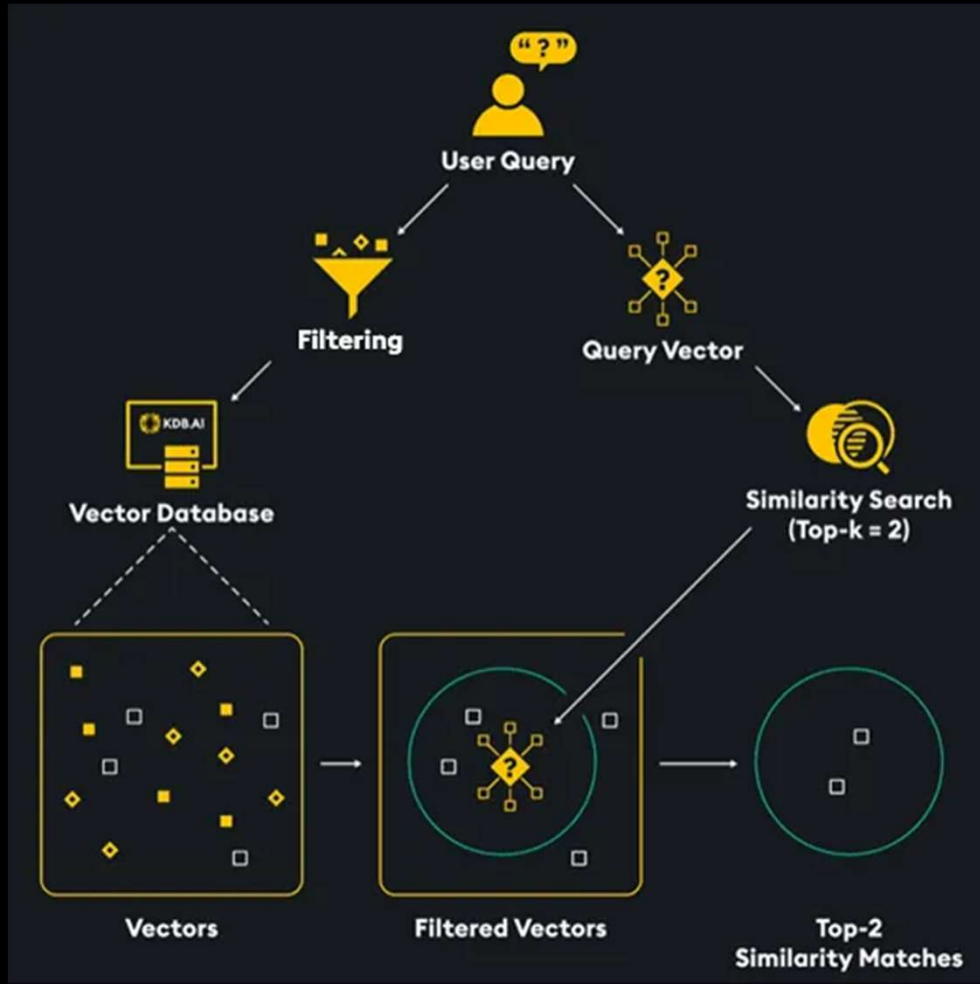


Hybrid Search

Metadata, keywords searched using syntactic search



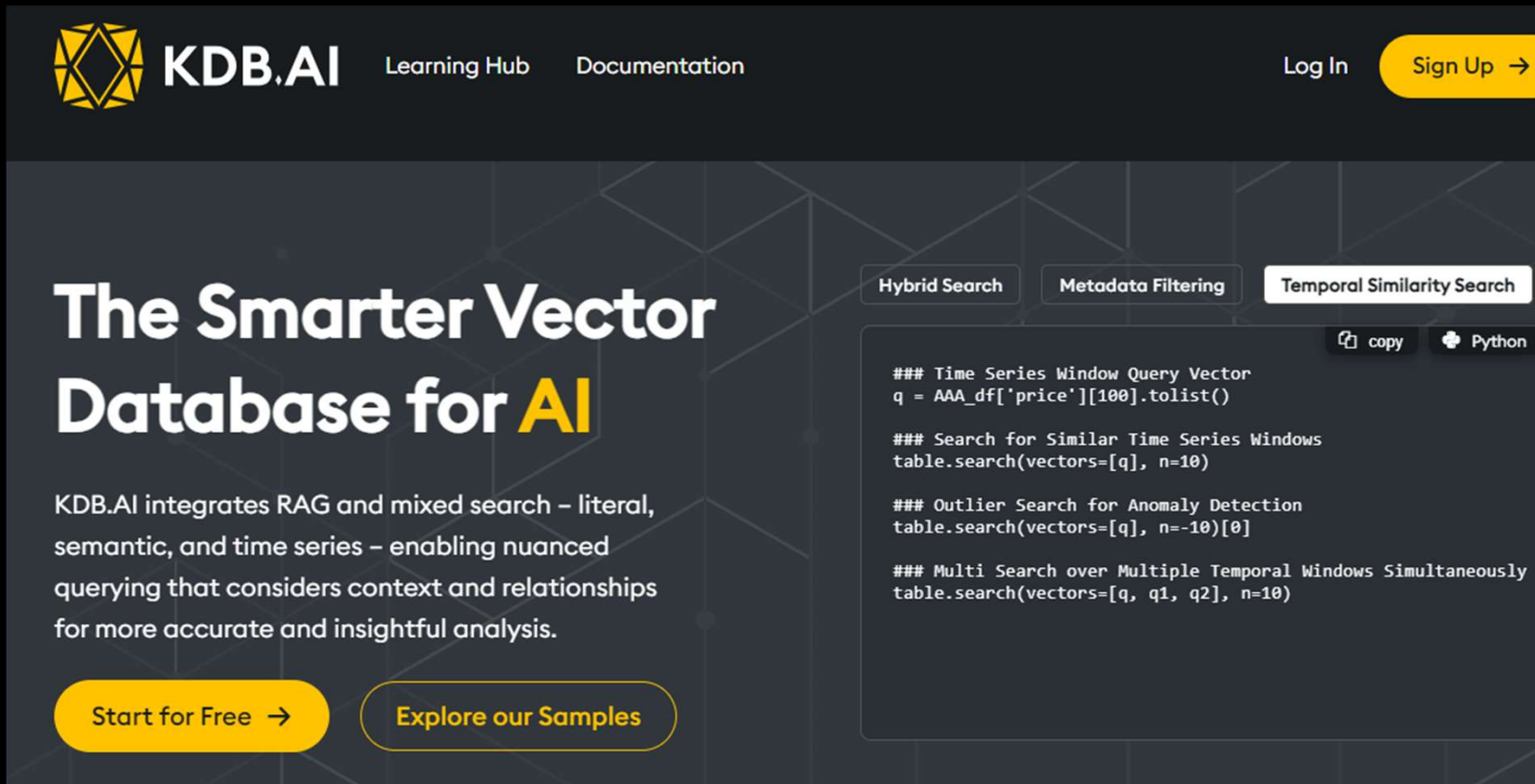
Retrieval optimization - Metadata filtering



Metadata examples include

Dates
Times
Genres
Categories
Names
Types
Descriptions
Etc.

Specialized vector databases



The screenshot shows the KDB.AI website homepage. The header features the KDB.AI logo, navigation links for 'Learning Hub' and 'Documentation', and buttons for 'Log In' and 'Sign Up'. The main content area is titled 'The Smarter Vector Database for AI' and describes the database's capabilities in integrating RAG and mixed search. It includes two call-to-action buttons: 'Start for Free' and 'Explore our Samples'. On the right, there are tabs for 'Hybrid Search', 'Metadata Filtering', and 'Temporal Similarity Search'. Below these tabs is a code editor showing Python code for time series window queries, similar time series searches, outlier detection, and multi-search over multiple temporal windows.

KDB.AI Learning Hub Documentation Log In Sign Up →

The Smarter Vector Database for AI

KDB.AI integrates RAG and mixed search – literal, semantic, and time series – enabling nuanced querying that considers context and relationships for more accurate and insightful analysis.

Start for Free → Explore our Samples

Hybrid Search Metadata Filtering Temporal Similarity Search

```
### Time Series Window Query Vector
q = AAA_df['price'][100].tolist()

### Search for Similar Time Series Windows
table.search(vectors=[q], n=10)

### Outlier Search for Anomaly Detection
table.search(vectors=[q], n=-10)[0]

### Multi Search over Multiple Temporal Windows Simultaneously
table.search(vectors=[q, q1, q2], n=10)
```

copy Python

<https://kdb.ai/>

Metadata filtering

Let's imagine that we have stored in our vector database a large number of experiences and leisure offers (Ex: surf classes, zip line, gastronomic route, etc.).

The description of the experience is what we have encoded, using our embedding model.

Additionally, each offer has 3 key values or metadata: Date, price and place.

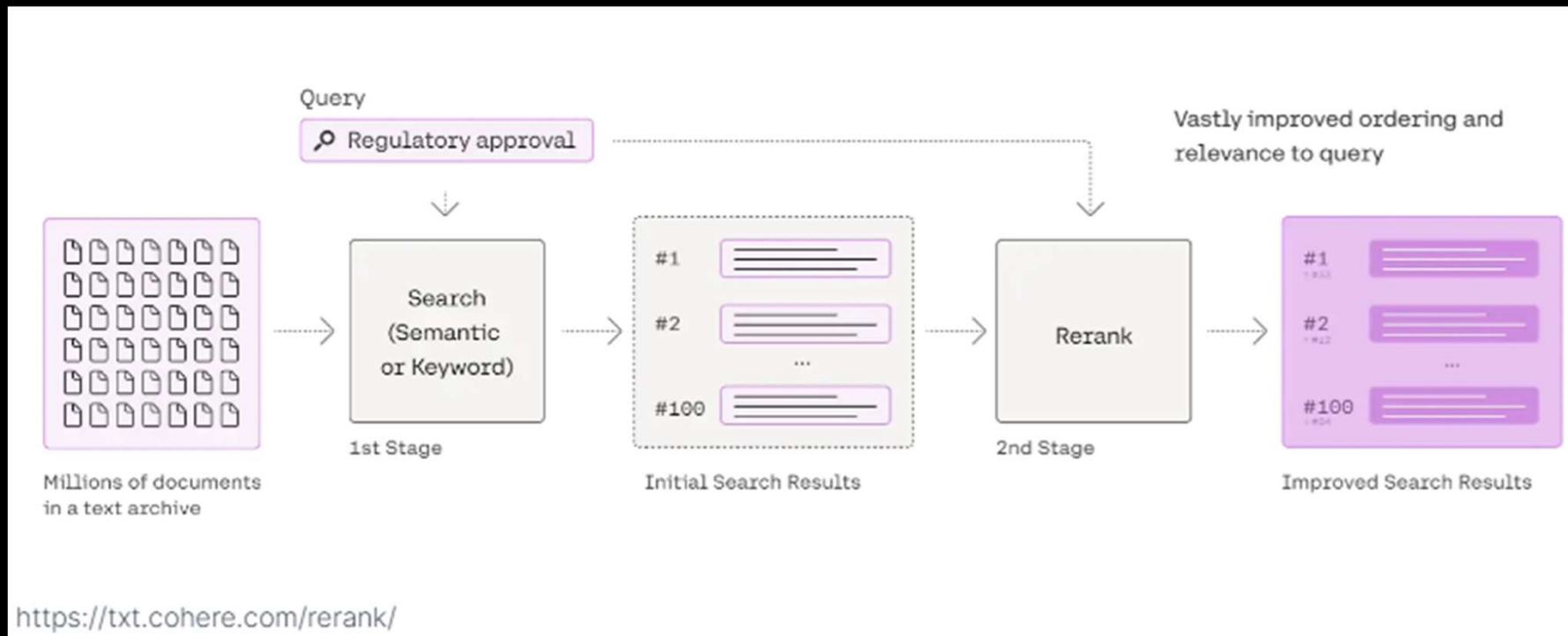
Let's imagine that a user is looking for an experience of this style: An experience in nature, that is for the whole family and safe. Furthermore, the price must be less than \$50 and the place is California.

Example of metadata filtering

```
#Search query specifying the query vector, top-k=3, and metadata filters
table.search(
  query_vector,
  n=3,
  filter=[("like", "Director", "George Lucas"), ("=", "ReleaseYear", "1977")]
)
```

The filtering greatly optimizes both the accuracy and efficiency of the similarity search, especially at scale.

Re-ranking



[advanced-rag-techniques/reranking.py](#) at main · pdichone/advanced-rag-techniques · GitHub

Query re-writing

- A query that mixes different semantic meanings won't yield a good semantic search result in a vector DB.

Data, tell me
about your
daughter Lal.
Also, isn't Star
Wars cool?

Query
Rewriter

Lal information

Compute
embedding
vector

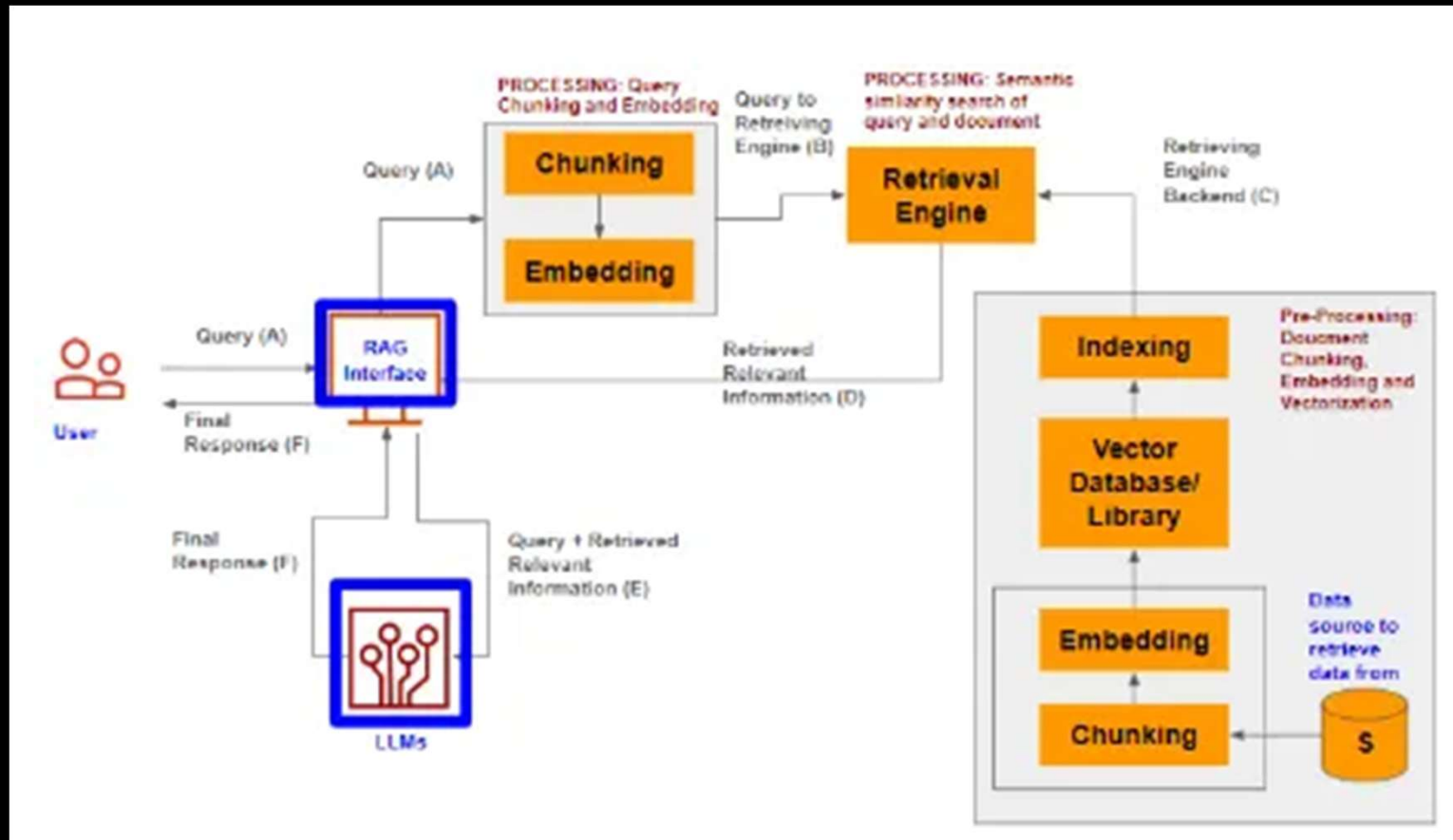
Vectordb of
Data's script
lines

Similar lines

Query re-writing

- Contextual Compression — Retrieval mechanism
- MultiQueryRetriever mechanism
- MultiVector Retriever Mechanism
- Time-weighted vector store retriever
(gives weightage to the time the document was accessed)
- Recursive or Iterative Retrieval
- Summary based retrieval
- Rule based retrieval
- Ensemble retriever

What steps can be tuned?



RAG Optimization knobs

Chunking

- Semantic chunking
- Agentic chunking

Indexing

- vector compression
- Metadata
- Hyperparameter tuning
- Choose right ANN algo

Query optimization

- Prompt engineering
- Query transformation, splitting
- Query expansion, compression
- Query routing, augmentation
- Lost in the middle problem*

Retrieval knobs

- Embedding models
- hybrid search weighting
- re-ranker models
- Multi-index search configurations
- Metadata filtering.
- Time-weighted retrieving
- Summary based retrieval
- Ensemble retriever

Generation knobs

- Choice of LLM
- Prompt
- LLM configuration

RAG pros and cons

- Faster & cheaper way to incorporate new or proprietary information into “GenAI” vs fine-tuning
- Updating info is just a matter of updating a database
- Can prevent “hallucinations” when you ask the model about something it wasn’t trained on
- If your boss wants “AI search”, this is an easy way to deliver it.
- Technically you aren’t “training” a model with this data
- You have made the world’s most overcomplicated search engine
- Very sensitive to the prompt templates you use to incorporate your data
- Non-deterministic
- It can still hallucinate
- Very sensitive to the relevancy of the information you retrieve

RAG hands-on

RAG with Google Vertex AI

Vertex AI Vector Search (Matching Engine – Self Controlled)

You create index + endpoint, you upsert vectors, you query neighbours.

A vector database you control (you do chunking, embeddings, upserts, retrieval logic)

Vertex AI Search / Discovery Engine (Fully Managed)

Managed datastore + search experience, different API/roles.

A managed “Google-quality” enterprise search system (Google handles indexing/ranking/search features; you integrate via APIs)

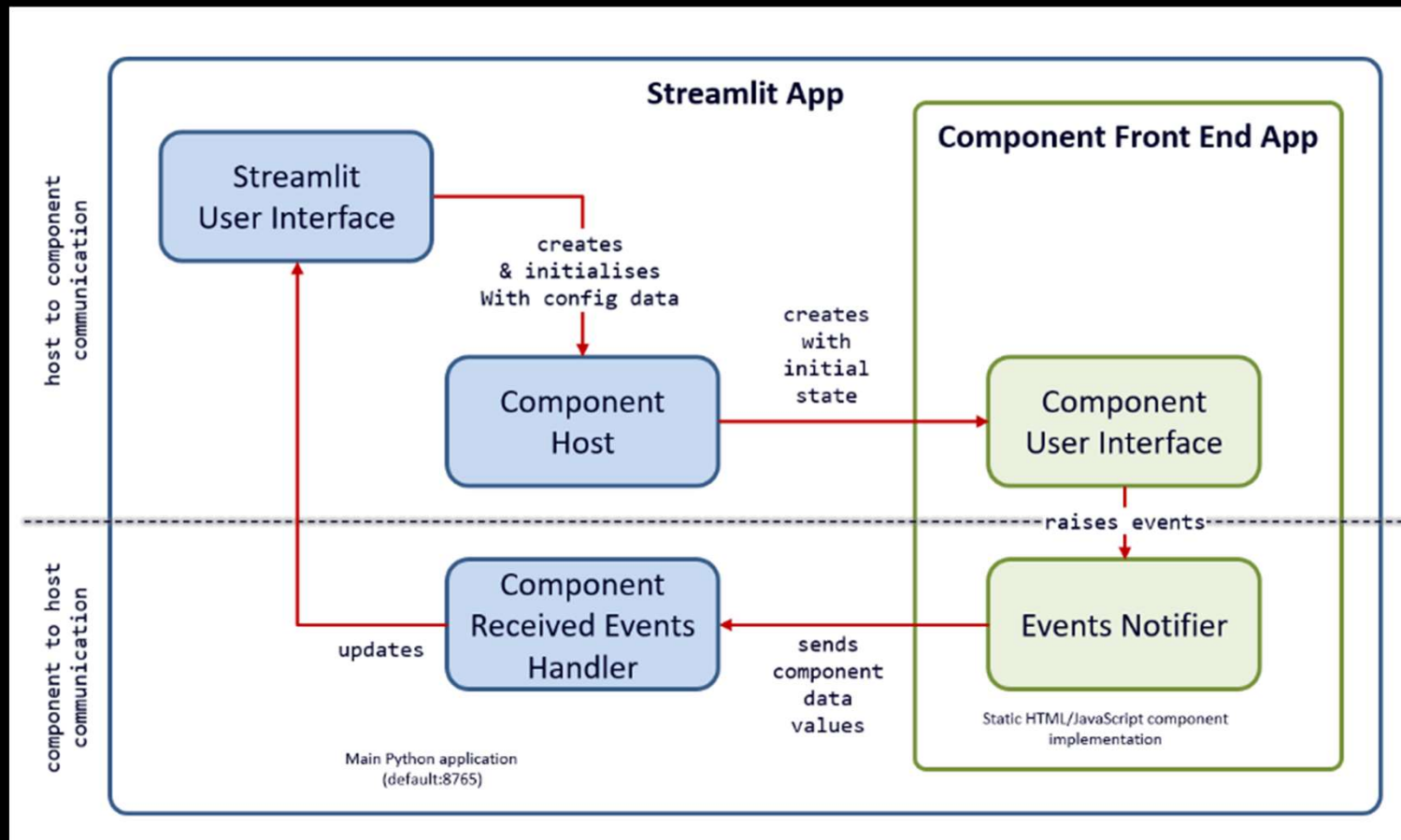
Vertex AI RAG Engine (Managed Infra)

Managed “RAG orchestration” layer (RAG Engine) that can use different retrieval backends and reduces infra plumbing.

Streamlit

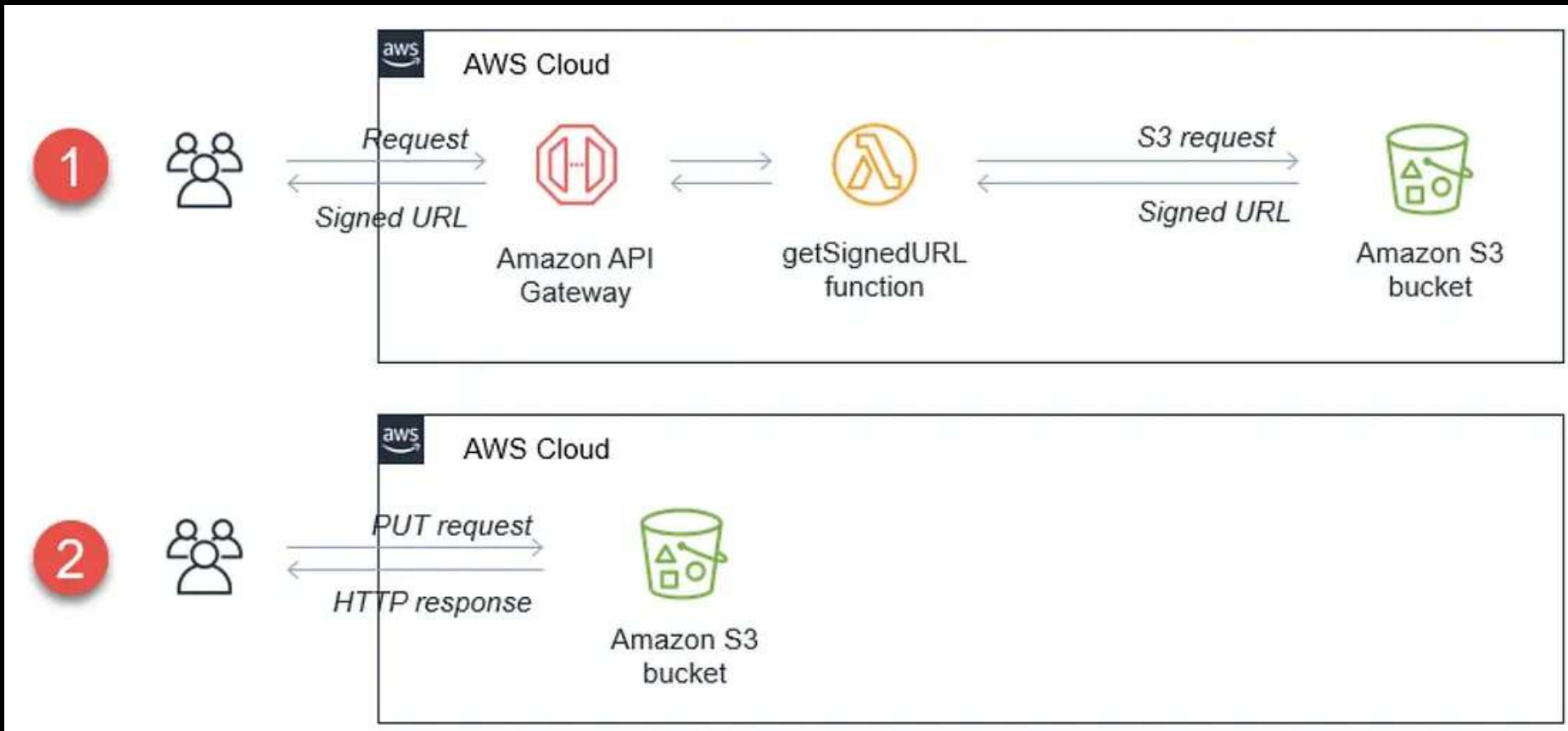
- Open-source app framework that helps create beautiful data apps in hours using pure python
- Acquired by Snowflake in 2022 for USD800M
- Aligns well with chatbot/chat based interfaces and integration with LLMs
- Well integrated and supported by popular frameworks and platforms such as Langchain, Huggingface etc.

Streamlit



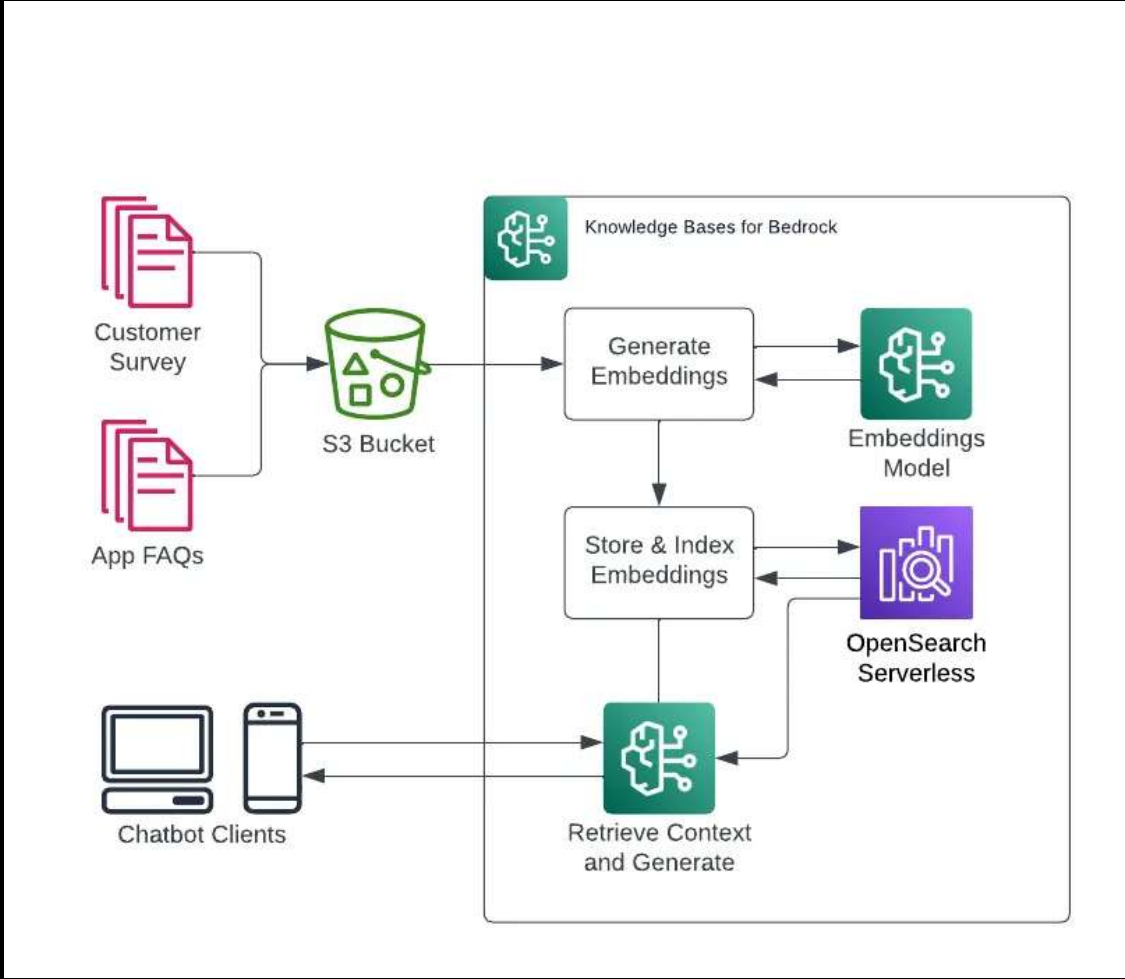
PDF chatbot with streamlit and Azure OpenAI

AWS S3

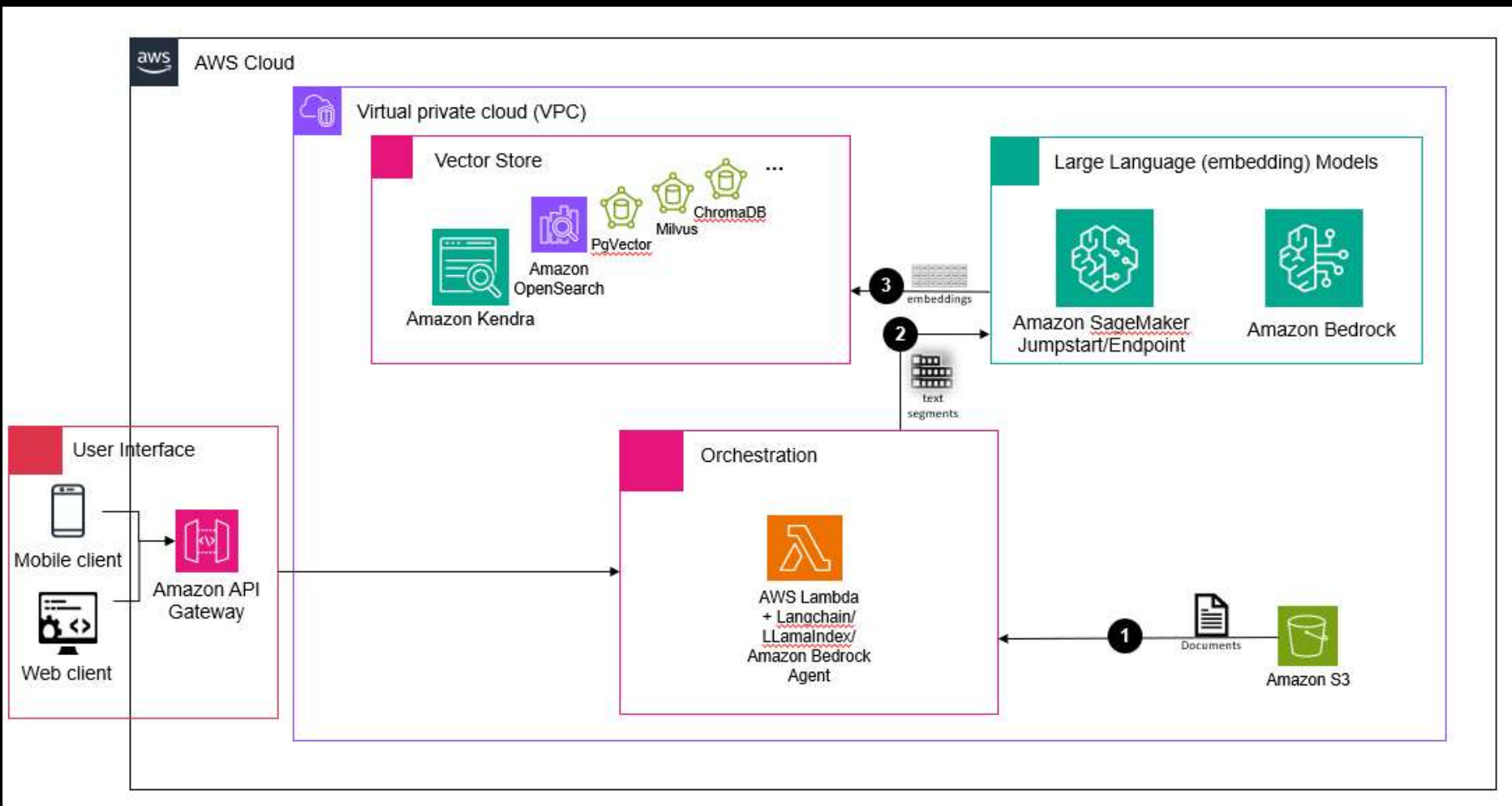


<https://aws.amazon.com/blogs/publicsector/use-modular-architecture-for-flexible-and-extensible-rag-based-generative-ai-solutions/>

Amazon Bedrock Knowledgebase

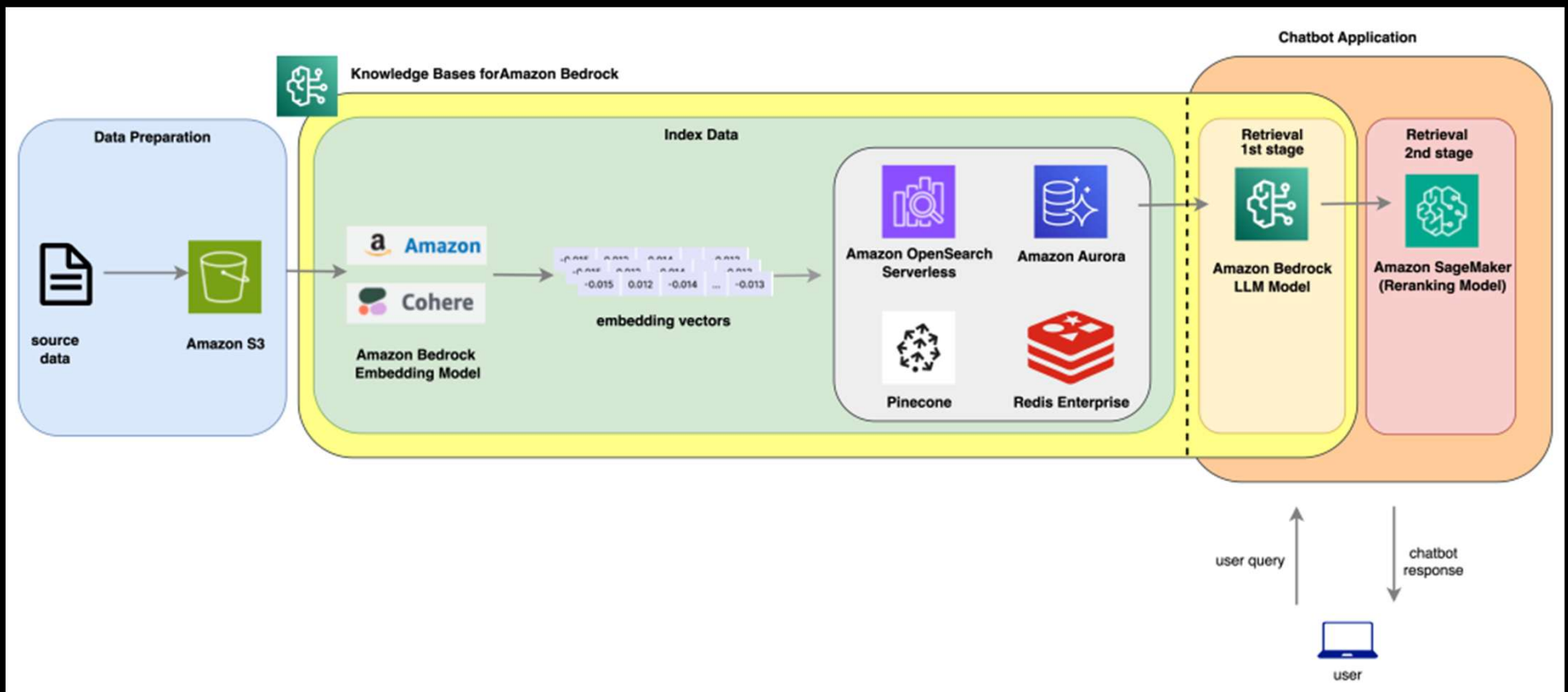


RAG with AWS



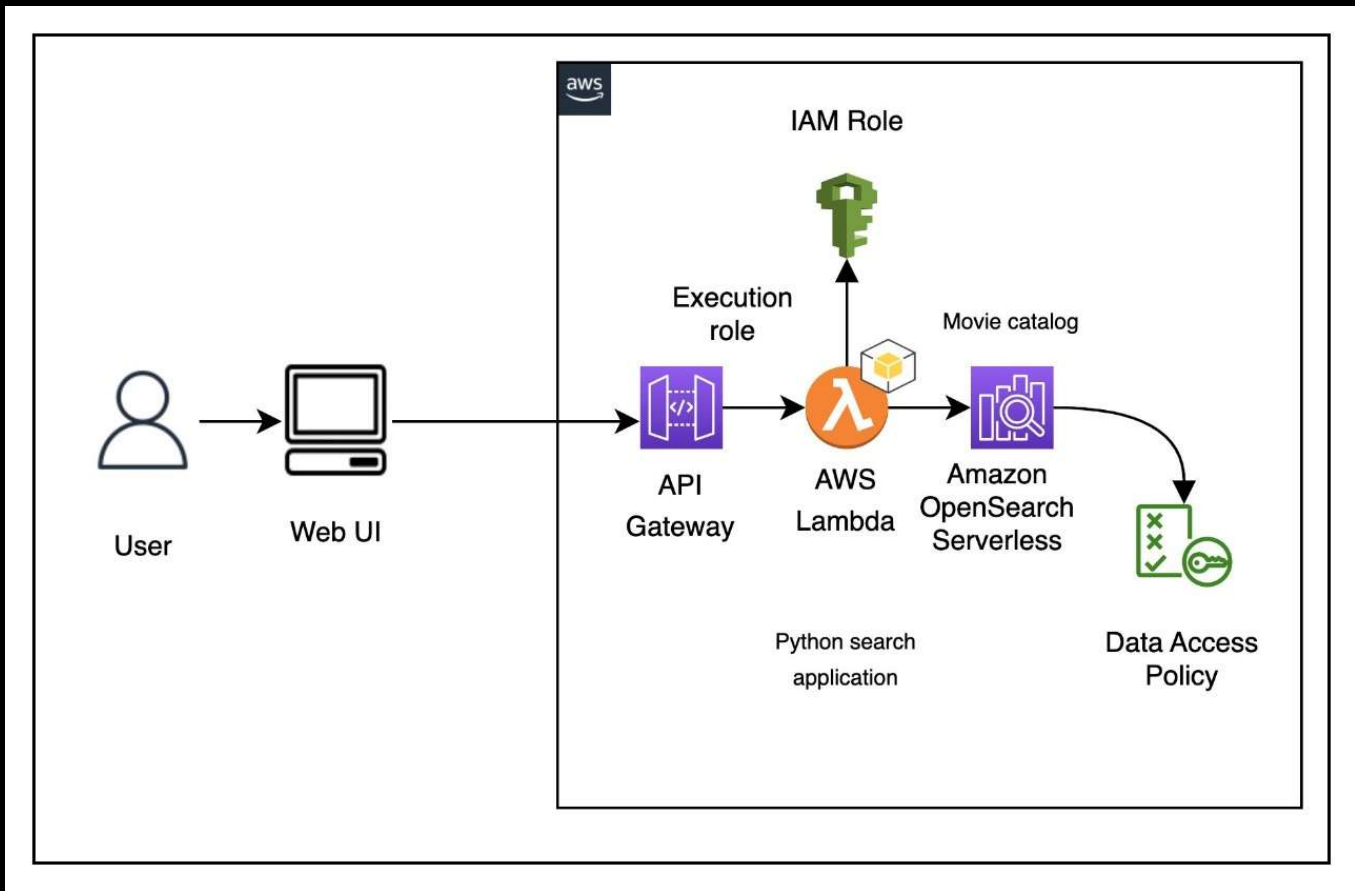
<https://aws.amazon.com/blogs/publicsector/use-modular-architecture-for-flexible-and-extensible-rag-based-generative-ai-solutions/>

RAG with AWS

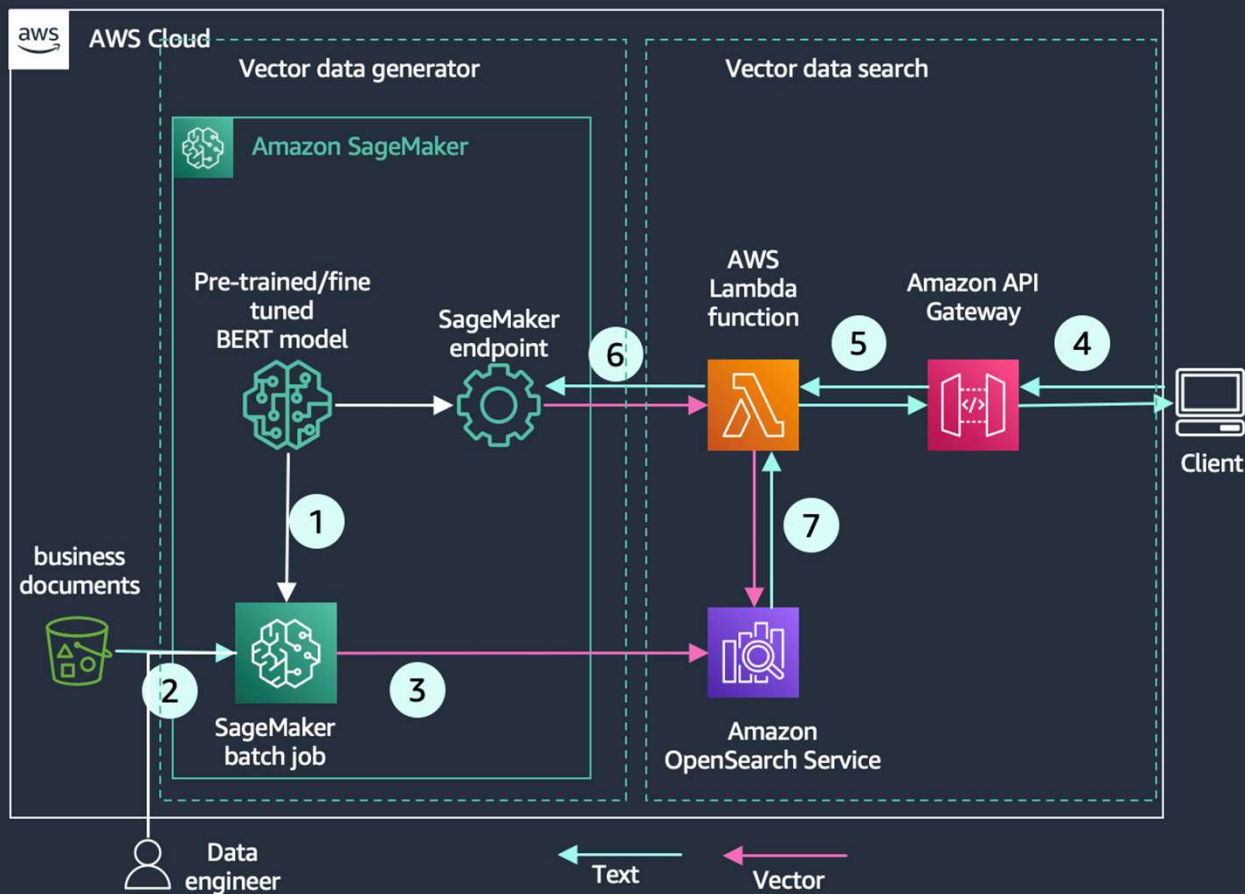


<https://aws.amazon.com/blogs/machine-learning/improve-ai-assistant-response-accuracy-using-knowledge-bases-for-amazon-bedrock-and-a-reranking-model/>

RAG with AWS



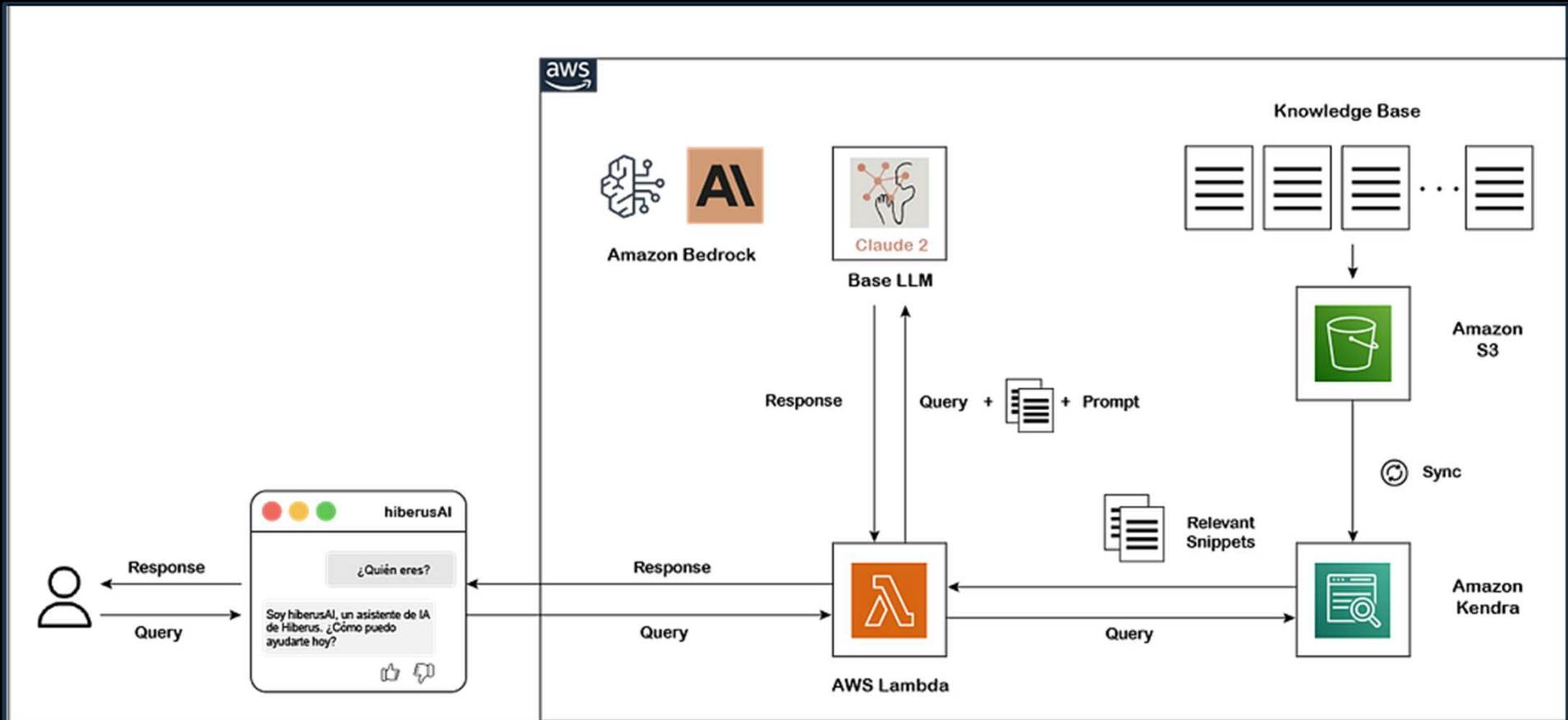
RAG with AWS



- 1 Data engineer load pre-trained or fine tuned BERT model into SageMaker
- 2 Data engineer run SageMaker batch job to generate vector for business documents with BERT
- 3 Store vector data into OpenSearch
- 4 Client submit search request to API Gateway
- 5 API Call Lambda backend service in Lambda
- 6 Backend service call SageMaker Endpoint to convert search query into vector
- 7 Use OpenSearch k-NN search to get semantic similar documents and return to client

<https://aws.amazon.com/blogs/big-data/amazon-opensearch-services-vector-database-capabilities-explained/>

RAG with Amazon Kendra



<https://aws.amazon.com/blogs/big-data/amazon-opensearch-services-vector-database-capabilities-explained/>

RAG with AWS

<https://itnext.io/aws-introduction-to-the-opensearch-service-as-a-vector-store-1b7cfb080691>

<https://levelup.gitconnected.com/rag-architecture-on-aws-integrating-opensearch-amazon-kendra-and-s3-for-smarter-retrieval-3c32572c426c>